软件工程(双语)

前言

本文档页码采用英文版第 9 版教材

题型

- 判断题（5题，共10分）
- 简答题（5题，共30分）来源于9张纸作业 课堂作业二
- 理解与沟通（3题，共30分）第七章
- 分析与设计（2题，共30分）第十章面向对象设计原则

复习提醒

5-8班老师：

1、前两道题，属于送分题，40分大概率得分都在36-40之间。判断题会有一种比较难以区分的，需要找到原句(时间比较充裕的情况下，可以根据语句，找到具体的章节，然后翻书看一下能不能找得到原句)，大家根据2/3的分配原则大概率也能拿到分。这种题大家不用过于纠结

2、剩下的五道大题，会从

（A）"名人名言"，

（B）画图题(用例图，顺序图，泳道图，活动图等抽一个，他不会让你们去创建什么主场景事件流，次场景事件流，直接画出图形就即可，大家根据PPT上的图，能够模仿画图就行)，

(C) 重构题(OCP，LSP等等等等，这种题一定是PPT或者课后练习题的原题，不会有改变)，

(D）白盒测试的计算题。

从里面抽5道题，可能会有不考的，可能会有的题型出两道。我能给的建议就是"从题干里找出关键词，定位在第几章，然后去翻书翻PPT）

3、不确定的情况下，我建议大家多写，写越多越好

4、不要空着，一定要写答案，会有辛苦分

1. 第一部分 软件与软件工程、软件过程（第1，2，3章）

1.1 软件的定义 P5

Software is:

1. instructions (computer programs) that when executed provide desired features, function, and performance;
2. data structures that enable the programs to adequately manipulate information;
3. descriptive information in both hard copy and virtual forms that describes the operation and use of the programs. (书本和ppt内容不一致，此处采用英九书本)

软件=程序+数据+文档

作业题1:

How does software differ from the artifacts produced by other engineering disciplines ?

- Software is both a product and a vehicle for delivering a product.
- As a product, software is an information transformer.
- As a vehicle for delivering a product, software serves as a basis for computer control, communication, and creation of other programs.

软件与其他工程学科产生的产物有何不同?

软件既是产品，也是交付产品的工具。

作为一种产品，软件是一个信息变换器。

作为产品交付的载体，软件作为计算机控制、通信及创建其他程序的基础。

作业题2:

How do software characteristics differ from hardware characteristics ?

- Software is developed or engineered it is not manufactured in the classical sense.
- Software doesn't "wear out" but is does deteriorate.
- Although the industry is moving toward component-based construction, most software continues to be custom-built.

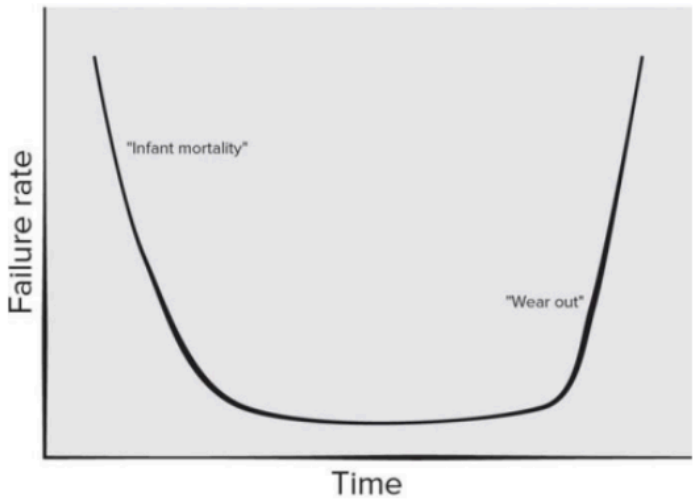
软件特性和硬件特性有何不同?

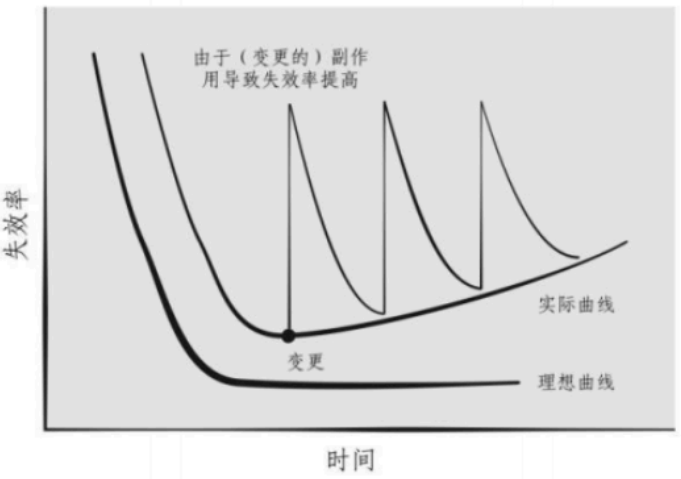
软件是被开发或工程化的，而不是传统意义上的制造。

软件不会“磨损”，但确实会退化。

尽管行业正向基于组件的构建方向发展，但大多数软件仍然是定制开发

1.2 软件退化（Software Deterioration） P6图片 软件特征硬件特征





> 软件和硬件失效率的解释：
>
> **硬件失效率曲线**（左图）：
>
> - 硬件的失效率随着时间的推移表现为一个“浴缸曲线”。
> - 初始阶段的高失效率（称为“婴儿死亡率”），这是由于初期故障较多。
> - 中期失效率较低且稳定。
> - 最后阶段失效率再次升高，表示硬件开始“磨损”并逐渐失效。
>
> **软件失效率曲线**（右图）：
>
> - 软件的失效率在初始阶段较高，主要由于早期的缺陷和错误。
> - 随着时间推移，失效率下降，但并不会像硬件那样达到一个恒定的低值，而是随着修改和更新（如图中标注的“更改”）可能再次升高。
> - 这种波动的失效率是由于每次修改可能引入新的错误，导致失效率上升，随后再下降到一个较低的水平，但不会完全稳定。

软件不会“磨损”，但确实会退化。退化的根本原因是变更。

1.3 软件工程（Software Engineering 简称 SE） P8

Software Engineering

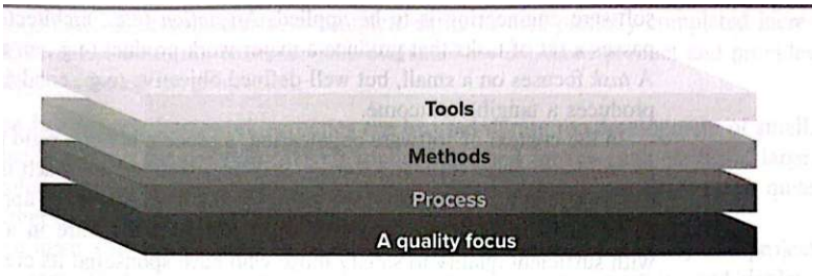
The IEEE definition：

① The application of a **systematic**, **disciplined**, **quantifiable** approach to the development, operation, and maintenance of software; that is, the application of **engineering** to software.

② The **study** of approaches as in ①.

软件工程定义2点 1系统化 2理论研究实践

软件工程三要素：过程、工具、方法，以质量为根基 P9最上面图片



书P9 软件工程层级

SE is a layered technology

The **bedrock** that supports **SE** is a **quality focus**.

process

- **framework** & **basis**, to hold the technology

methods

- provide the **technical** (**principles**) to build software

tools

- **support** (**aided**) for the process and the methods

1.4 软件过程（Software Process） P9最下面从1.3标题开始都是

讲了过程（Process）和过程框架（Process framework）两个概念，过程有三个层次，过程框架有五个框架活动和一组普适性活动（umbrella activities）

框架活动：沟通（communication）、计划（planning）、建模（modeling）、构建（construction）部署（deployment），是和产生软件产品直接相关的

普适性活动是管理的，和软件产品生产不直接相关，贯穿于整个项目过程（applied throughout a software project）

1. A **process** is defined as a set of **activities**, **actions**, and **tasks**

- An **activity** achieves a **broad** objective.
- An **action** produces a **major** work product.
- A **task** focuses on a **small**, but **well-defined** objective that produces a **tangible** outcome.

三者关系：Activity（活动）⊃ Action（动作）⊃ Task（任务）

1. A **process framework** includes **five framework activities** and a set of **umbrella activities**.

- A **framework activity** contains some **actions**, each **action** is defined as a set of **task sets**.
- each **task set** encompasses a series of **“the work tasks, the work products, the quality assurance points and the milestones”**

软件过程

2、软件工程的实过程施框架定了 5 个框架活动与 8 个普适性活动，每一项活动都有其专有的动作、任务和内涵。但软件工程不应该是教条的法则，不要求项目团队机械执行，可以根据项目要解决的问题、特点与团队文化等因素，进行适应性调整，**请根据自己的理解，简单描述每一个框架活动的任务（6 分），并谈一下不同项目的实施过程，应从哪些方面来调整（9 分）。**

21-22-2中文真题

1.5 软件工程实践的7条原则 P14-15标题 斜体不要写

1. **存在理由（The Reason It All Exists）：** 为用户提供价值。
2. **保持简洁（KISS）：** 设计应尽可能简单。
3. **保持愿景（Maintain the Vision）：** 保持概念完整性。
4. **关注下游（What You Produce, Others Will Consume）：** 考虑到维护者和后续开发者。
5. **面向未来（Be Open to the Future）：** 为变更和通用问题做准备。

> 例如女士养宠物的例子中，目前女士提出的是养猫，但是如果只实现养猫的话，不具有可扩展性，即无法面向未来，所以应该实现宠物的抽象类，这样以后女士提出养狗、鸟等不同宠时，不用修改太多，这里就是用到了多态的概念

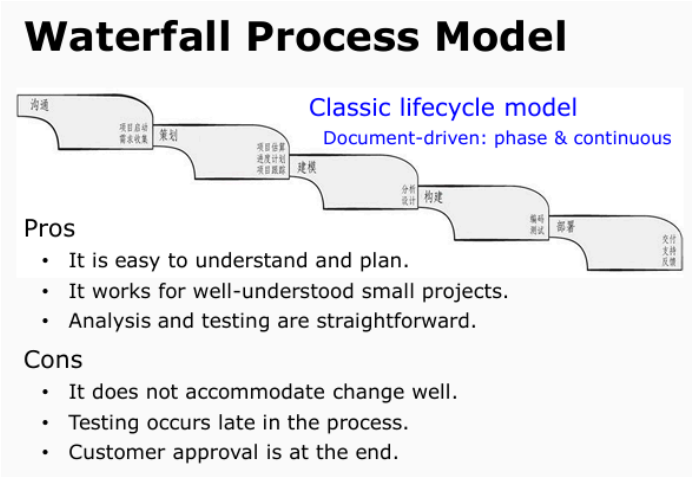
6. **计划复用（Plan Ahead for Reuse）：** 提高系统的长期价值。

> 包括知识复用（分析结果复用，即分析模型）、方法复用（设计结果复用，设计模型）、（代码复用），继承是共性局限性大（共性东西少），结构化关联

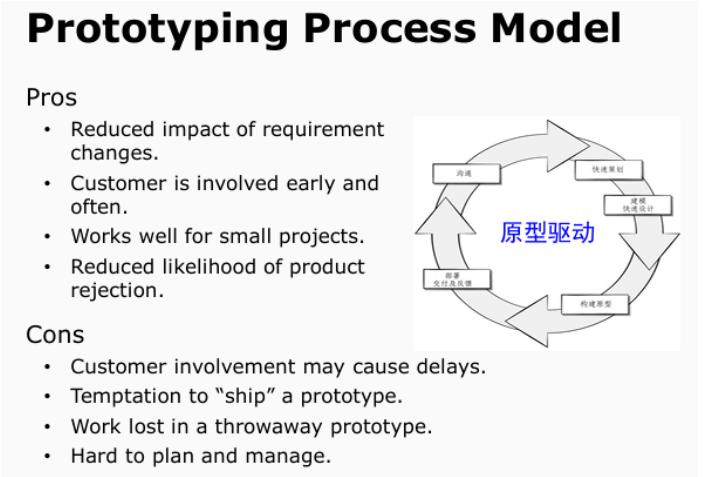
7. **思考（Think!）：** 在行动前进行清晰、完整的思考

1.6 常用过程模型(Prescriptive Process Models)的特点 P26 P30, 38

有**常用过程模型**和**敏捷过程模型**两大类，每类4个，这里是常用过程模型4个



瀑布模型 图P26



原型过程模型 图P27

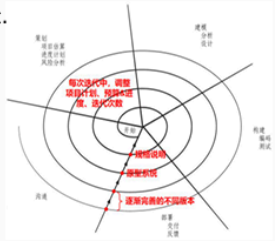
Spiral Process Model

演化模型

风险驱动：瀑布与原型的结合

Pros

- Continuous customer involvement.
- Development risks are managed.
- Suitable for large, complex projects.
- It works well for extensible products.



Cons

- Risk analysis failures can doom the project.
- Project may be hard to manage.
- Requires an expert development team.

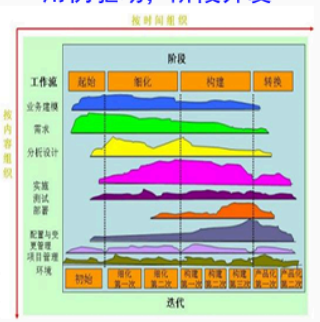
螺旋模型 图P30

Unified Process Model

用例驱动，阶段并发

Pros

- Quality documentation emphasized.
- Continuous customer involvement.
- Accommodates requirements changes.
- Works well for maintenance projects.



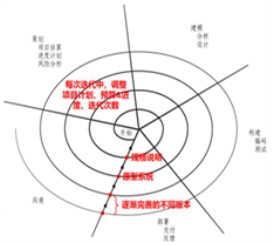
Cons

- Use cases are not always precise.
- Tricky software increment integration.
- Overlapping phases can cause problems.
- Requires expert development team.

统一过程(UP)模型 P32有UP图 非此图

Why the spiral model is the best approach to software development in a modern context?

- timelines for the development of modern software are getting shorter and shorter
- customers are becoming more diverse and making the understanding of requirements even harder
- changes to requirements are becoming even more common before delivery.



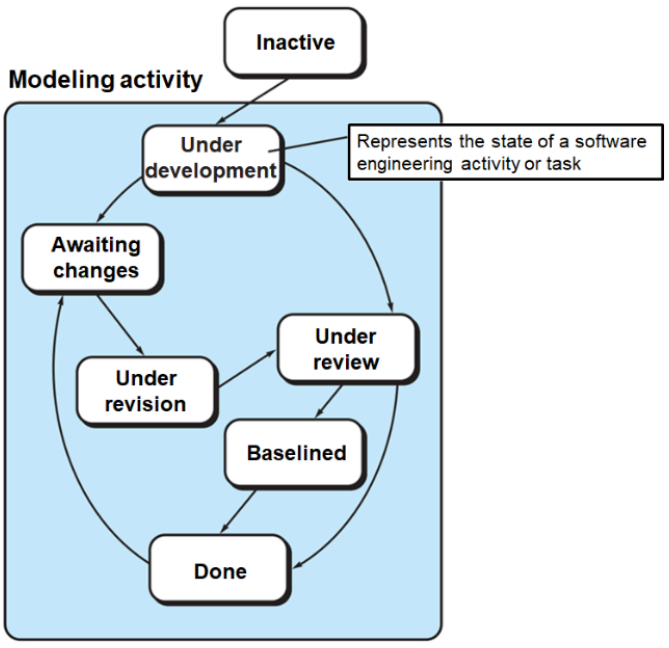
- need a way to provide incremental or evolutionary delivery to better adapt to uncertainty, allows the delivery of partial solutions in an orderly and planned manner, most importantly, reflects what really happens when complex systems are built.

为什么螺旋模型是现代背景下软件开发的最佳方法

为什么螺旋模型是现代背景下软件开发的最佳方法?

现代软件开发的生命周期/时间线变得越来越短。客户变得越来越多元化，这使得理解需求变得更加困难。在交付之前，需求的变更变得更加普遍。需要一种能够提供增量式或演进式交付的方法，以更好地适应不确定性。

2、Among the numerous software process models, the concurrent development model is often more appropriate for product engineering projects where different engineering teams are involved. As shown in the figure, it describes any iterative elements and concurrent elements in the development process. Please illustrate the workflow of the model with an example and talk about your understanding. （15 pts）



21-22-2双语真题 并发开发模型

1.7 敏捷(Agility)概念的理解 P38

🎉 **敏捷宣言**

Agile Manifesto

A process model is working, it is necessary to adopt a realistic mechanism to ensure discipline while treating software engineers with tolerance.

What are the tradeoffs proposed by the "Agile Manifesto"?

1. individuals and interactions over processes and tools,
2. working software over comprehensive documentation,
3. customer collaboration over contract negotiation, and
4. responding to change over following a plan

Agile is not antithetical to solid SE practice, and can be applied as an overriding philosophy for all software work.

中文翻译

一个过程模型在起作用时，有必要采用一种现实的机制来确保纪律，同时以一定程度宽容对待软件工程师（产生可控的错误）。

“敏捷宣言”提出的权衡是什么?

1. 个体和互动胜过过程和工具，
2. 可工作的软件胜过详尽的文档，
3. 客户协作胜过合同谈判，
4. 响应变化胜过遵循计划。

敏捷并不与稳固的软件工程实践对立，并且可以作为所有软件工作的指导理念。

🦄 **敏捷哲学**

与上面的宣言一一对应

agile philosophy

What are the key issues stressed by an agile philosophy?

1. Emphasis on rapid delivery of software that satisfies the customer
2. Recognition that change represents opportunity
3. Communication and collaboration between team members and customers
4. The importance of self-organizing teams

中文翻译

敏捷哲学强调的关键问题是什么?

1. 强调快速交付满足客户的软件
2. 认识到变化代表着机会
3. 团队成员与客户之间的沟通和协作
4. 自组织团队的重要性

1. **What is Agility（来自PPT）**

- Effective (rapid and adaptive) response to change.
- Effective communication among all stakeholders.
- Drawing the customer onto the team
- Organizing a team so that it is in control of the work performed
- Rapid, incremental delivery of software.

2. **What are the key issues stressed by an agile philosophy?（来自PPT）敏捷哲学所强调的关键问题是什么?**

- Emphasis on rapid delivery of software that satisfies the customer
- Recognition that change represents opportunity
- Communication and collaboration between team members and customers
- The importance of self-organizing teams

1.8 敏捷过程基本特征 P40

What is an Agile Process?（来自PPT）敏捷过程是什么

adaptive agile **incremental** development strategy

1. Driven by customer requirement description (scenarios).
2. Customer **feedback** is frequent and acted on.
3. Recognizes that plans are short-lived.
4. Develops iteratively with a heavy emphasis on construction.
5. Delivers multiple ‘software increments’ as executable prototypes.
6. Adapts as project or technical changes occur.

下文 Scrum P42、XP P46、Kanban P48、DevOps P50 四种主流敏捷落地框架，均是敏捷过程的具体实现形式，各自的流程环节、工作步骤属于敏捷过程细分特征

2. 第二部分 建模（第6，7章）

2.1 需求工程的6项任务（作用）+根基 P86 6.1和6.2节小标题

6项任务分别是6.1.1到6.1.7小标题：起始 (Inception)、获取 (Elicitation)、细化 (Elaboration)、协商 (Negotiation)、规格说明 (Specification)、确认 (Validation)

6.2节讲根基 建立根基的6个步骤(6.2.1到6.2.6六个小标题)：

1. **识别利益相关者 (Identifying Stakeholders)：** 找出所有直接或间接从系统中获益的人。
2. **识别多重观点 (Recognizing Multiple Viewpoints)：** 考虑到不同利益相关者的需求可能存在冲突或不一致。
3. **协同合作 (Working Toward Collaboration)：** 鼓励利益相关者之间以及与开发团队之间的合作。
4. **提出第一批问题 (Asking the First Questions)：** 关注客户及其他利益相关者，了解问题的整体目标和收益。
5. **非功能性需求 (Nonfunctional Requirements/NFR)：** 识别质量属性（如安全性、性能等）和系统约束。
6. **可追溯性 (Traceability)：** 建立需求与软件工程其他工作产品（如设计、测试用例）之间的文档化联系

🥖 **首次提问(P90-91)**

> 三组问题，第一部分是目标与收益，第二部分是解决方案，第三部分是效率
>
> 注意考试会这三种问题各给出一种，问这个问题的重点和意义是什么，答案就是加粗部分的
>
> 特点：上下文无关（context-free）这些问题是**通用的、抽象的、不依赖于任何特定业务领域或技术细节**

> **The first set of questions focuses on the customer and other stakeholders and the overall project goals and benefits.**
>
> - Who is behind the request for this work?
> - Who will use the solution?
> - What will be the economic benefit of a successful solution?
> - Is there another source for the solution that you need?
>
> **These questions help to identify all stakeholders who will have interest in the software to be built, and identify the measurable benefit of a successful implementation and possible alternatives to custom software development.**

> **The second set of questions focuses on a solution.**
>
> - How would you characterize "good" output that would be generated by a successful solution?

- What problem(s) will this solution address?
- Can you show me (or describe) the business environment in which the solution will be used?
- Will special performance issues or constraints affect the way the solution is approached?

These questions enable you to gain a better understanding of the problem and allows the customer to voice their perceptions about a solution.

The third set of questions (meta-questions) focuses on the effectiveness of the communication activity itself.

- Are you the right person to answer these questions? Are your answers "official"?
- Are my questions relevant to the problem that you have?
- Am I asking too many questions?
- Can anyone else provide additional information?
- Should I be asking you anything else?

These questions will help to "break the ice" and initiate the communication that is essential to successful elicitation.

1、At the beginning of the project, in order to elicit requirements, the following three questions were asked to customers and other stakeholders. Please try to analyze what the goal of each question is (9 pts) and what effect it has on the scheme selection of system construction (6 pts) ?

1) Who is behind the request for this work?
2) What problem(s) will this solution address?
3) Can anyone else provide additional information?

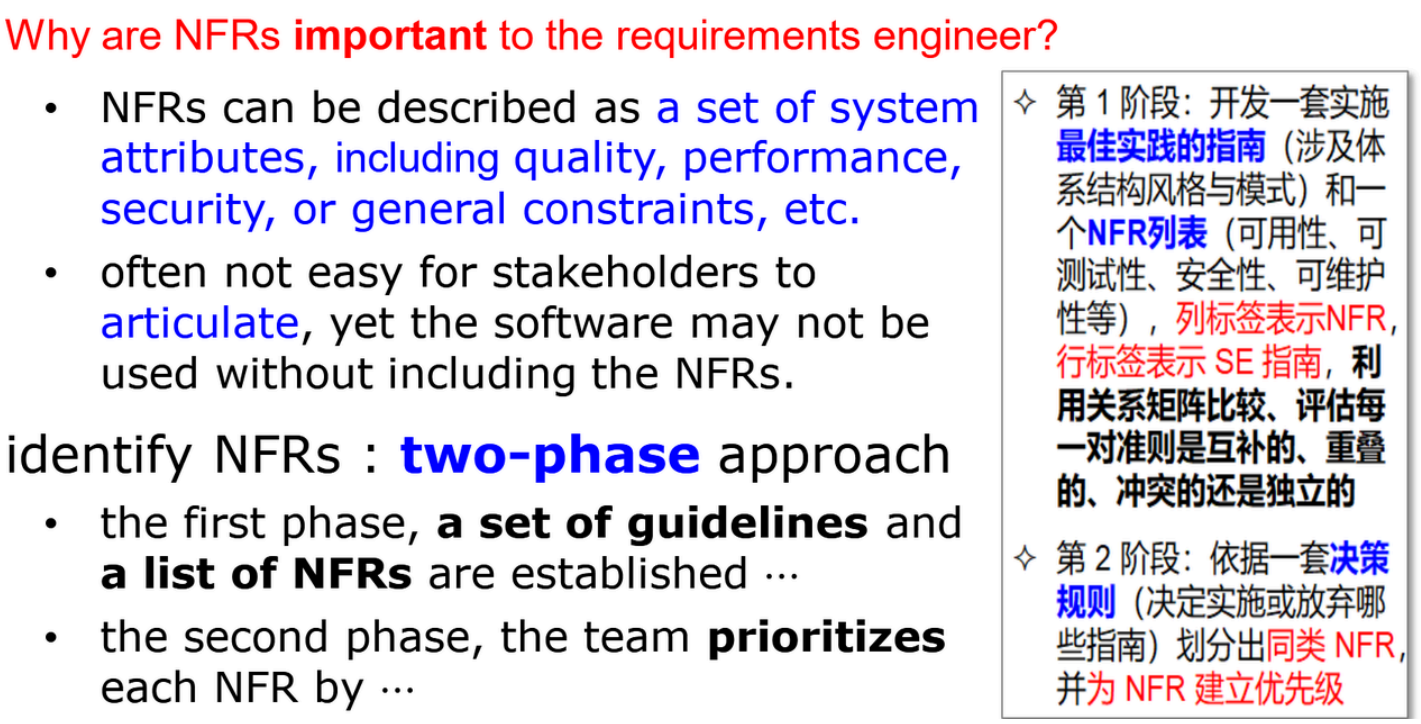
21-22-2双语真题

1、项目开始时，为了导出需求，向客户和其他利益相关者提问了如下 3 个问题，**请尝试分析每个问题的目标是什么（9 分），对系统构建的方案选择有什么作用（6 分）？**

1) 成功的解决方案将带来什么样的经济效益 ？
2) 如何描述由成功的解决方案产生的“良好”输出的特征 ？
3) 你是回答问题的合适人选吗 ？

21-22-2中文真题

非功能性需求P91为什么对工程师很重要



课堂作业二题目三

非功能性需求是决定软件质量的关键维度（性能P245、安全性、可靠性P246）

工程师（尤其是需求工程师）通过协作来识别**公共区域（Areas of commonality）矛盾区域（Areas of conflict or inconsistency）** P90框出来的INFO段落

2.2 两种需求建模的方法和模型元素

2.2.1 结构化分析 (Structured Analysis) P110脚注1为其定义

结构化分析主要体现为**流导向建模 (Flow-Oriented Modeling)**。

其中处理过程将数据作为独立实体加以转换。数据对象建模定义了对象的属性和关系，操作数据对象的处理建模应表明当数据对象在系统内流动时处理过程将如何转换数据。

模型元素： 虽然本书的当前章节不再详细展开，但它传统上包括数据流图 (DFD) 和数据字典等工具。

2.2.2 面向对象分析 (Object-Oriented Analysis, OOA) P109 (Section 7.1 概览)；基于场景建模详见P112；基于类建模（核心 OOA 内容）始于P119

面向对象分析在教材中主要通过**基于场景的建模**和**基于类的建模**来体现。

这种方法关注类的定义和影响客户需求的类之间的协作方式。

核心模型元素：

- **基于场景的元素：** 如**用例 (Use Cases)**、用例图、活动图和泳道图，从“参与者”的角度描述需求。

- **基于类的元素：** 如**类图 (Class Diagrams)**、属性、操作、协作以及 CRC 模型（类-责任-协作），定义了问题空间中的实体及其行为

2.3 用例描述、用例图、活动图P133-134、泳道图P135

PPT UNIT3 25页往后

2.3.1 用例描述P96-99（初步开发）P112-118(详细文档化)、用例图P101/119

❤️ **用例图 (Use Cases Diagram)**

用例图由四部分组成，分别是参与者，用例，系统边界和关系。画图时，一般按照：系统边界、参与者、用例和关系连线的顺序画。

1. **参与者 (Actor)**

- **概念**：在系统之外，与系统进行交互的人、外部系统、定时任务或硬件设备。
- **画法**：通常用一个小人（Stick Figure）表示。如果是外部系统，也可以用带有 `«actor»` 关键字的矩形框表示。
- **注意**：参与者是“角色”，而不是具体的某个人（例如“管理员”是一个 Actor，而“张三”不是）。

2. **用例 (Use Case)**

- **概念**：系统能够为参与者提供的一项完整的功能或服务。
- **画法**：一个**椭圆**，里面写上用例的名称。
- **命名规范**：通常采用**动宾短语**，如“登录账号”、“提交订单”、“查询余额”。

3. **系统边界 (System Boundary)**

- **概念**：用例图所研究的系统范围，用来区分哪些功能在系统内，哪些在系统外。
- **画法**：一个**大矩形框**，系统名称写在框的顶部或内部边缘，所有的**用例**都放框里面，**参与者**都放框外面。

4. **关系**

- 连接参与者与用例、或者用例与用例之间的线。

关联关系 (Association)

- **含义**：参与者与用例之间的通信路径，表示该参与者可以使用这个功能。
- **画法**：一条**普通的实线**（有时带箭头指向用例，不带箭头亦可）。

包含关系 (Include)

- **含义**：一个复杂的用例（基础用例）**必须**包含另一个用例（包含用例）的行为。基础用例执行时，包含用例一定会执行。

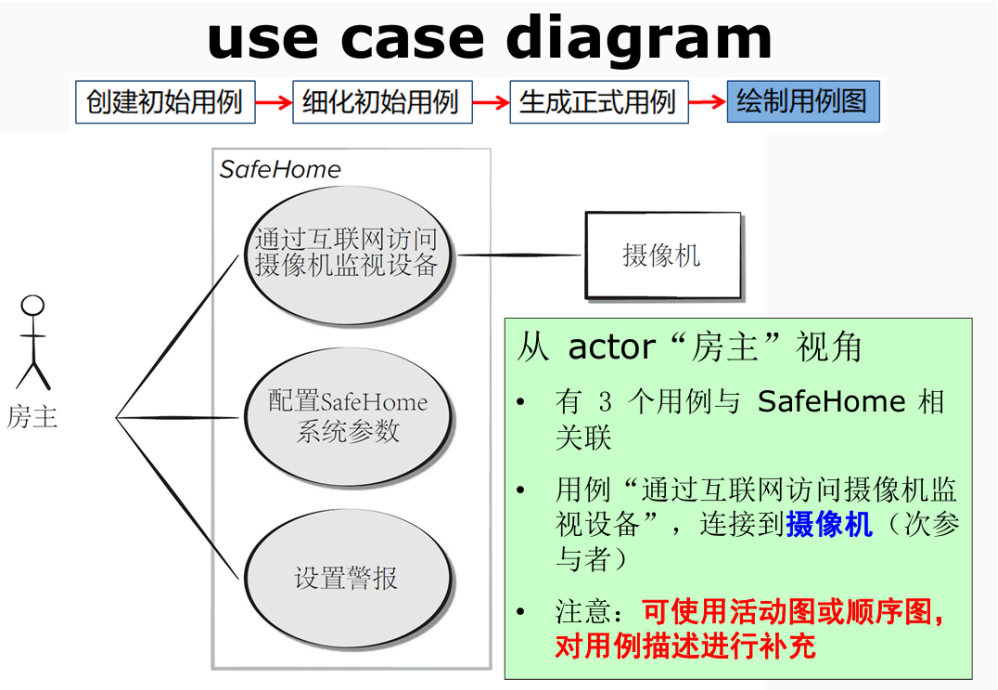
- **画法：虚线箭头 + `«include»` 关键字**，箭头由基础用例指向被包含的用例。
- **例子**：执行“修改密码”或“支付订单”时，系统**必须**先执行“身份验证”。

扩展关系 (Extend)

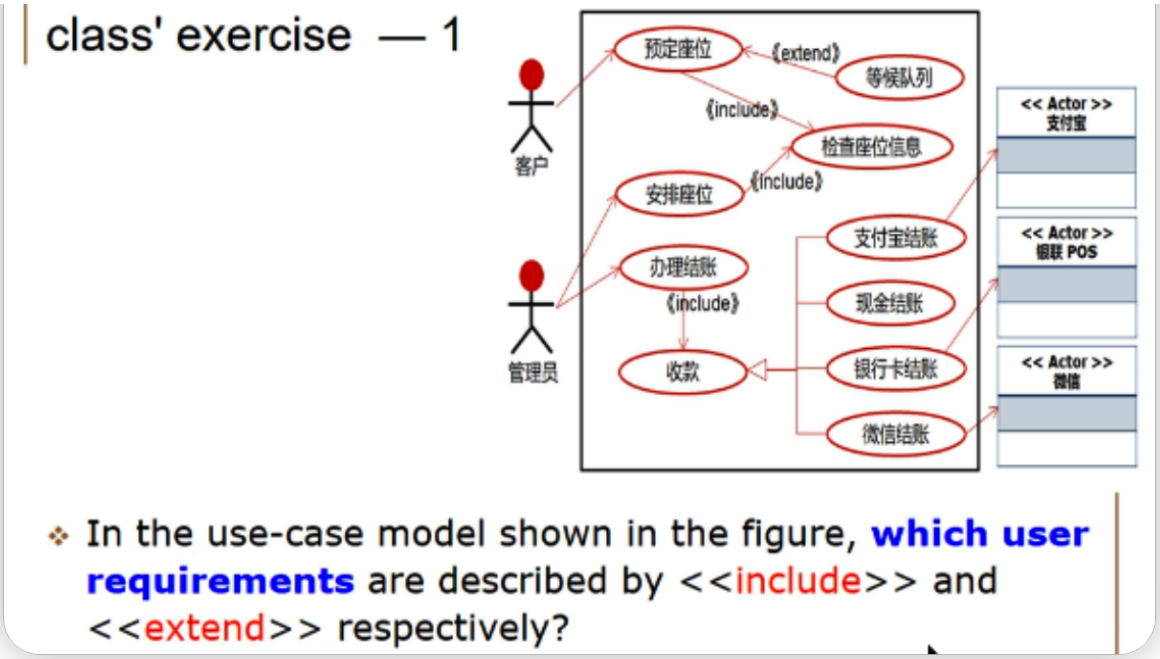
- **含义**：一个用例（扩展用例）在**特定条件满足时**，可以为另一个用例（基础用例）添加新的行为（可选的）。
- **画法：虚线箭头 + `«extend»` 关键字，注意：箭头由扩展用例指向基础用例**（逆向指向）。
- **例子**：在“提交订单”时，如果用户有优惠券，可以执行“使用优惠券”；没有就不执行。

泛化关系 (继承)

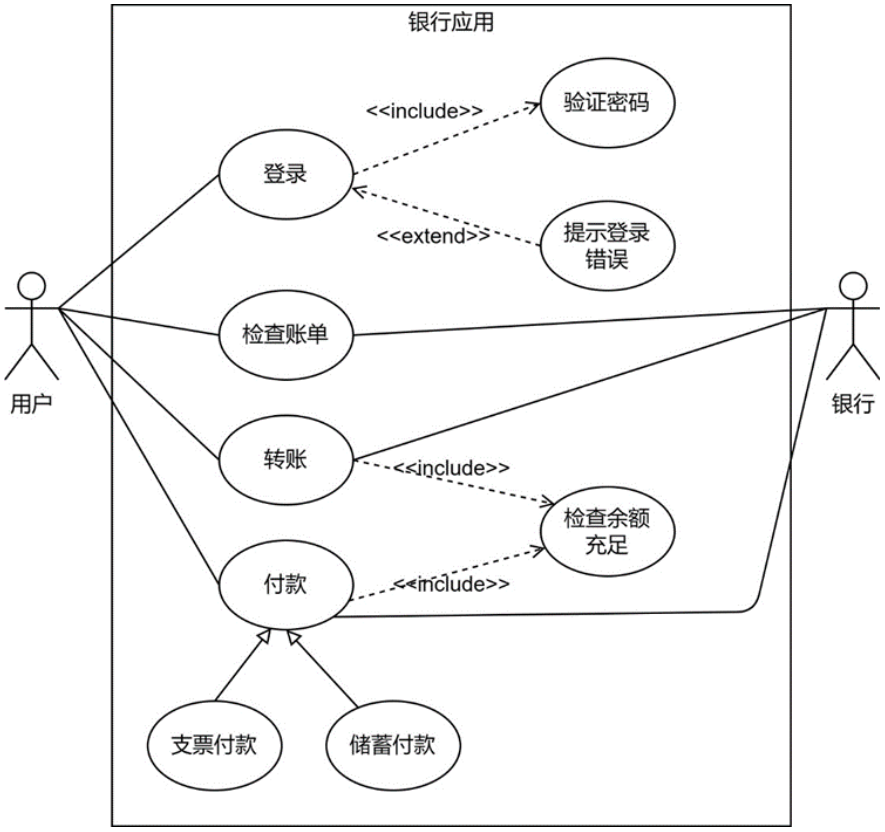
- **含义**：分为参与者的泛化和用例的泛化。表示一般与特殊的关系。
- **画法：实线 + 空心闭合三角形**，箭头指向父类（更通用的角色或用例）。



课件中用例图画法



课堂作业一 题目一第二问



用例图示例

`<<include>>` 核心用例 → 核心用例引出的子用例。表示强制性的包含关系。当执行事件 A 时，必然触发事件B。用于提取公共功能和行为。

`<<extend>>` 扩展出来的可选用例 → 基础用例。表示可选的扩展关系。当执行事件 A 且满足特定条件时，可以选择执行事件 B。用于描述可选行为和条件行为。

2.3.2 活动图(泳道图)P133-134

活动图 (Activity Diagram)

活动图由六部分组成，分别是起点与终点、活动状态、动作流、分支与合并、分叉与汇合、以及泳道。画图时，一般按照：泳道划分（可选）、起点、活动与动作流、条件分支与并行控制、终点的顺序画。

1. **起点与终点 (Initial and Final Nodes)**

控制流的开始位置与结束位置。一张活动图必须有且仅有一个起点，但可以有零个或多个终点。

起点：一个实心黑圆点。

终点：一个内部带实心黑圆点的圆圈（同心圆）。

流程终止点（仅销毁当前当前分支流，不结束整个活动）：一个内部带“X”的圆圈。

2. **活动状态 / 动作 (Activity / Action Nodes)**

流程中所经历的步骤、操作或业务处理过程。

画法：一个**圆角矩形**，里面写上具体的动作或步骤描述。

命名规范：通常采用**动宾短语**，如“填写表单”、“校验库存”、“发送通知”。

3. **动作流 (Control Flow)**

连接各个活动状态，表示动作执行的先后顺序。

画法：一条**带箭头的实线**，箭头指向下一个将要执行的活动。

4. **分支与合并 (Decision and Merge Nodes)**

分支（Decision）：根据不同的条件（Guard Conditions）走向不同的流程路径。

合并（Merge）：将多条互斥的分支路径重新汇聚到主流程中。

画法：一个**空心菱形**。条件写在菱形外面。

- **分支**：一条进入线，多条移出线。每条移出线上用中括号 `[条件]` 标注触发条件（如 `[密码正确]`、`[密码错误]`）。
- **合并**：多条进入线，一条移出线，线上不需要写条件。

5. **分叉与汇合 (Fork and Join Nodes)（并行控制）**

分叉（Fork）：一个流程同时拆分为多个**并行**执行的子流程（没有先后顺序，同时进行）。

汇合（Join）：所有并行的子流程全部执行完毕后，才能继续向下执行后续流程。

画法：一条**加粗的水平或垂直实心线段（同步条）**。

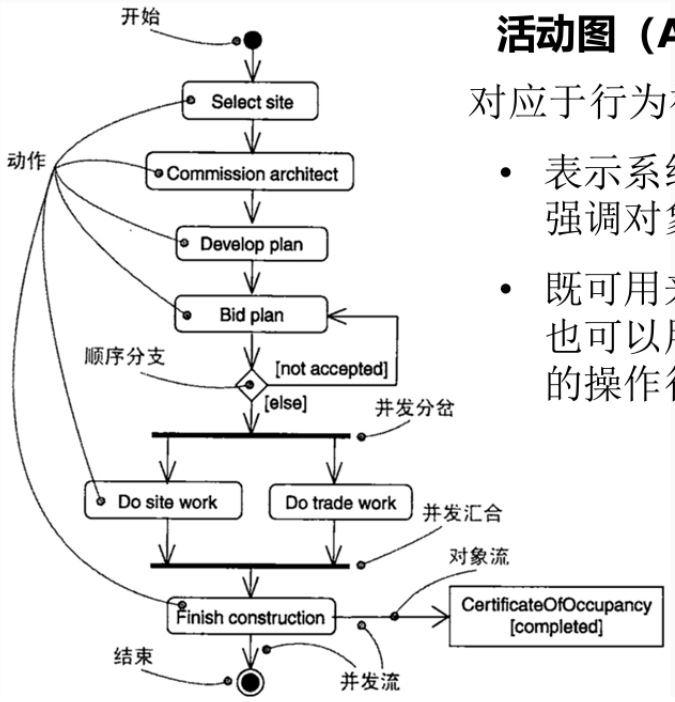
- **分叉**：一条进入线，多条移出线。
- **汇合**：多条进入线，一条移出线。

6. **泳道 (Swimlanes)**

用来明确某一个活动具体是由**哪一个角色、部门或系统组件**来执行的。将业务流程的职责划分得更清晰。

画法：用**垂直或水平的实线**将绘图区域横向或纵向隔开，形成若干个长条形区域（类似游泳池的泳道），每个泳道顶部写上角色或系统名称。相关的活动绘制在对应的泳道内。

Functional Modeling



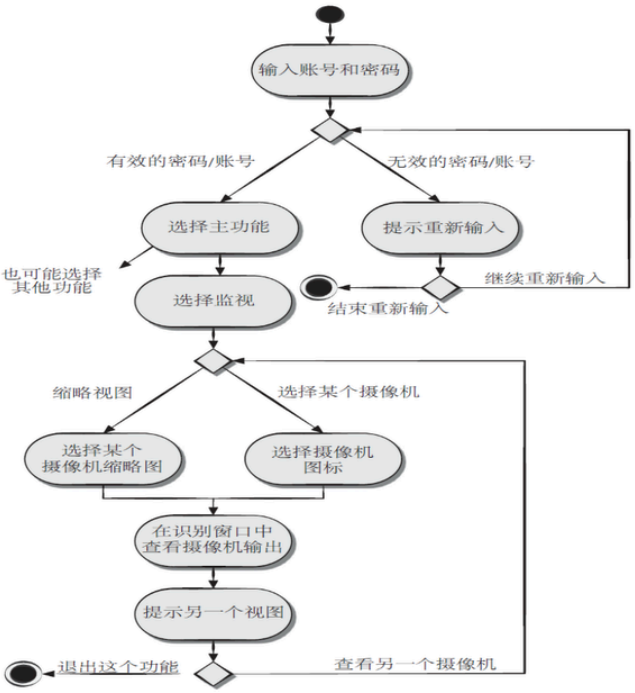
活动图（Activity Diagram）

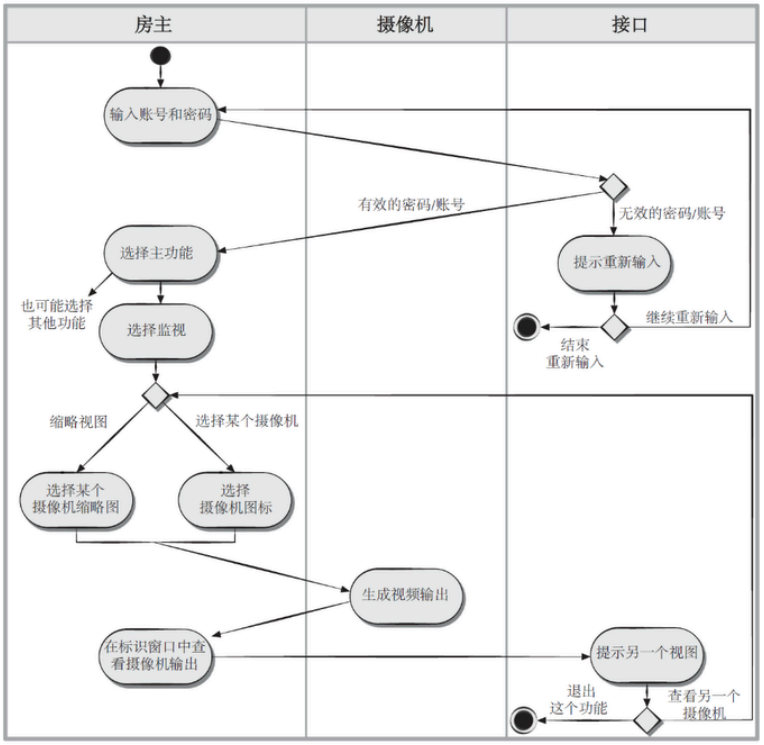
对应于行为视图

- 表示系统中各种活动的次序，强调对象之间的控制流程
- 既可用来描述用例的工作流程，也可以用来描述类中某个方法的操作行为

after Scenario-Based Modeling …

用**活动图**（**泳道图**）、顺序图，**提供特定场景内交互流的图形化表示，来补充用例**





课件中活动图和顺序图作为场景建模补充用例图

3、Since the suppliers of products are not in the same city, when you place an order as shown in the figure on an e-commerce platform through the shopping cart, the system needs to provide the function of splitting the order into some delivery orders. Please model this function with UML activity diagram. （10 pts）

序号	商品编号	商品名称	供应商地址
01	C01	华为手机	上海
02	S03	打印纸	北京
03	D01	滑板	广州
04	N01	牛仔裤	北京
05	C02	iPad	上海
……	……	……	……

21-22-2双语真题

2.4 类图、类之间的关系 P96 必考

教材部分类图页码(都不完整)：P102,P123,P125,P152(聚合关系),P158(接口)

🍰 类图 (Class Diagram)

类图由四部分组成，分别是类、接口、包和关系。画图时，一般按照：识别类与接口、填充属性与方法、梳理类间关系连线、标注多重性的顺序画。

类 (Class)

- **概念**：对具有相同属性、方法和关系的对象的抽象。
- **画法**：一个**三层矩形框**。
  - **顶层**：类名（如果是抽象类，类名用*斜体*表示）。
  - **中层**：属性（成员变量）。
  - **底层**：方法（成员函数）。
- **可见性修饰符**：
  - `+` ：Public（公开）
  - `-` ：Private（私有）
  - `#` ：Protected（受保护）
  - `~` ：Package（包内可见）
- **命名与格式规范**：
  - 属性：`可见性 属性名：类型 = 默认值`（如 `- age: int = 18`）。
  - 方法：`可见性 方法名(参数名：参数类型)：返回值类型`（如 `+ getAge(): int`）。

接口 (Interface)

- **概念**：一种特殊的类，只包含操作的声明而不包含具体实现，用以定义行为契约。
- **画法**：
  - **常规画法**：类似于类矩形框，但在顶层类名上方带有关键字 `«interface»`。
  - **棒棒糖画法**：一个**小圆圈 + 一条实线**，圆圈旁写上接口名（通常在不需要展示接口方法时使用）。

包 (Package)

- **概念**：用于对类图中的类进行分组和组织，类似于代码中的文件夹或命名空间。
- **画法**：一个**左上角带标签的矩形框**（像一个文件夹图标），标签内或矩形内部写上包名。

关系种类	核心含义	UML 图形特征
关联 (Association)	类与类之间的结构性关系（has-a），通常是长期的（如成员变量）。可以是单向或双向	实线 + 常用箭头（或无箭头双向）如果是单向关联，持有者->被持有者
聚合 (Aggregation)	整体与部分的关系（has-a），但部分可以离开整体而独立存在（弱拥有）	**实线加空心菱形，整体<>->部分，菱形在整体端，箭头指向被包含的部分**
组合 (Composition)	也是整体与部分的关系，但部分不能离开整体而存在，生命周期一致（强拥有）	**实心菱形，注意多重性，需要标数字**
泛化 (Generalization)/ 继承(Inheritance)	父类与子类的关系	**实线 + 空心三角形箭头（箭头指向父类）**
依赖 (Dependency)	一个类用到了另一个类（use-a），通常是临时性的（如方法参数、局部变量）	虚线 + 常用箭头（箭头指向被依赖的类）使用者->被使用者
实现	类与接口的关系	虚线 + 空心三角形箭头（箭头指向接口）

Class-Based Modeling

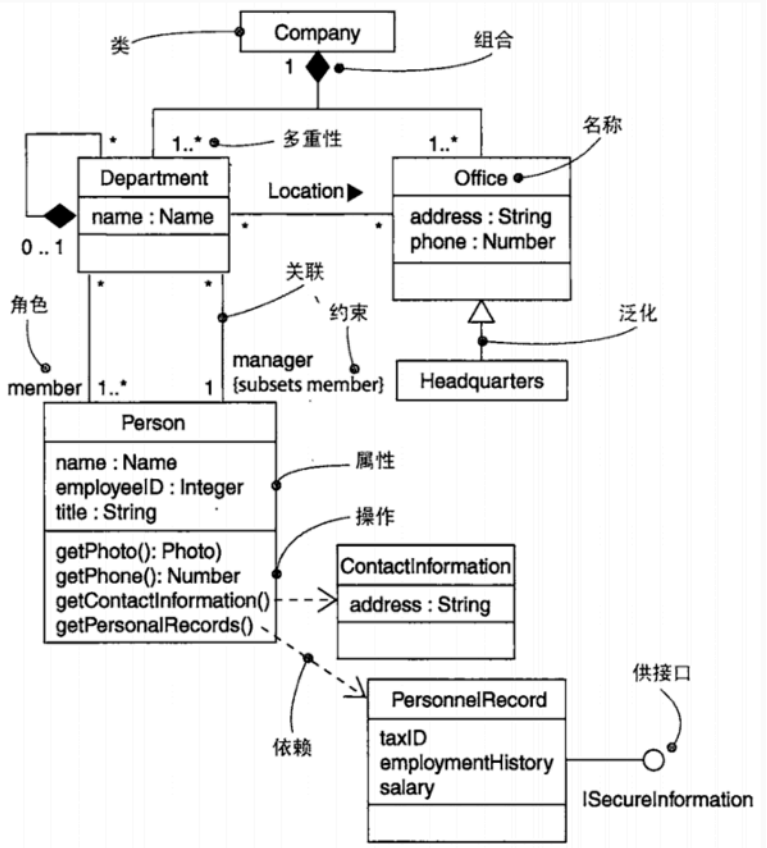
类图（Class Diagram）

对应于结构视图

- 描述系统的静态结构
- 类图建模类和类之间的关系，描述系统内所声明的类
- 没有描述系统运行时类的行为

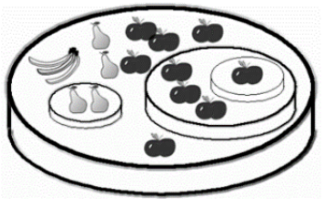
类间关系

- 泛化（继承）
- 关联 & 依赖
- 实现



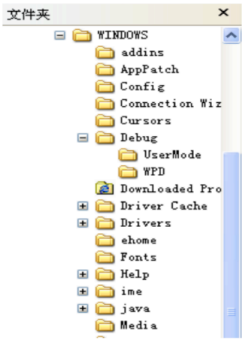
课件中类图

2、The combination of fruit plates is shown in the figure. How will you design and develop this system? Please use the class diagram to give the design model and explain the reasons for choosing the scheme.？（10 pts）



21-22-2双语真题

2、windows 的资源管理器如图所示，若由你来系统实现，你将如何设计？**请用类图来给出设计方案（7 分），并阐述方案选择的理由（3 分）？**



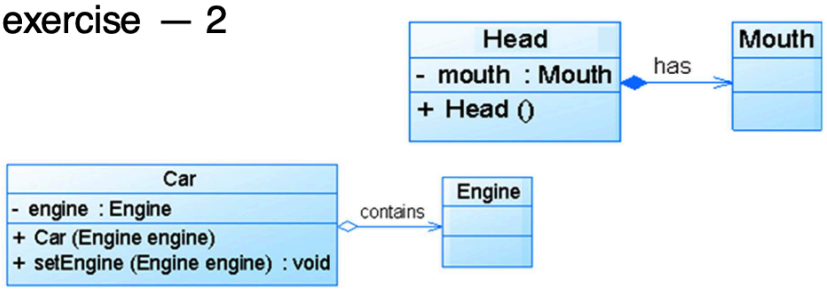
21-22-2中文卷真题

Multiplicity

0..1	zero to one (optional)
n	specific number
0..*	zero to many
1..*	one to many
m..n	specific number range

多重性数字标记

exercise — 2

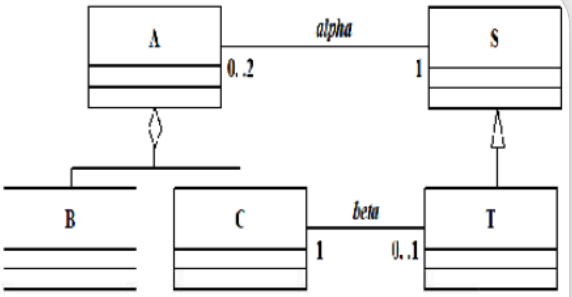


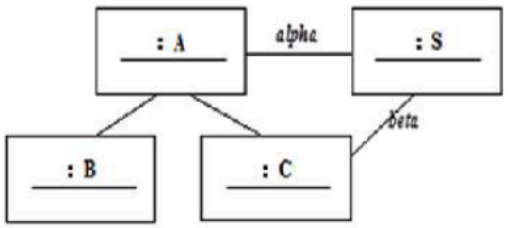
1. **What are the meanings of the symbols "◇⟶" and "◆⟶" respectively ? Using the class model example shown in the diagram, explain the difference between the two?**

课堂作业一第二题

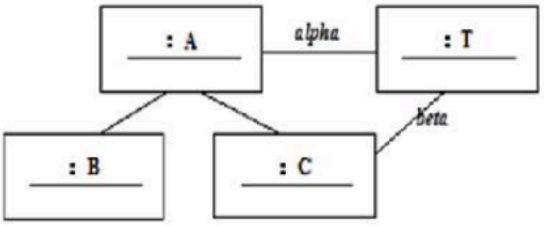
exercise — 3

- The right diagram is a class model.
- The following diagrams (1) and (2) are two instances of class model during program runtime.





(1)

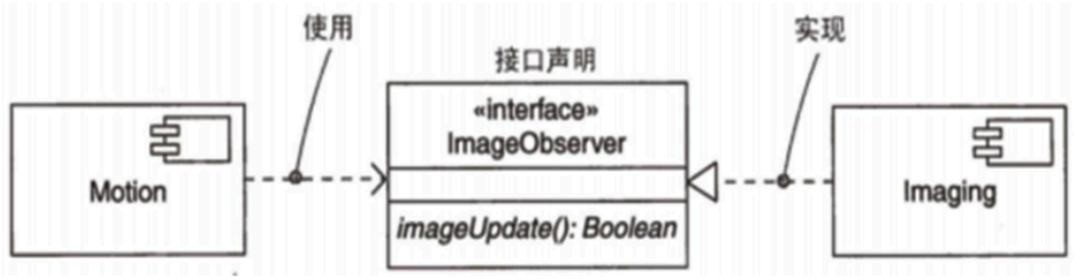


(2)

Which one is correct ? state your reasons.

课堂作业一第三题

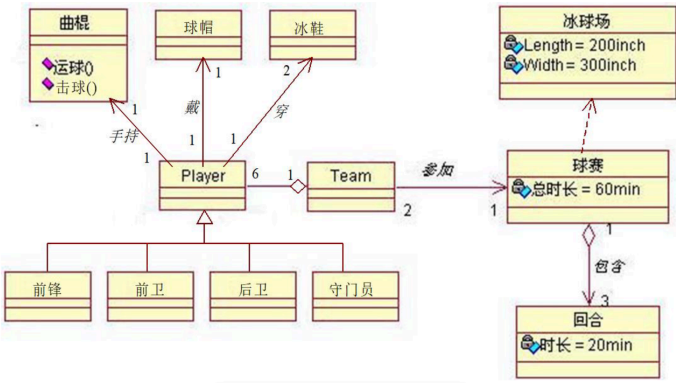
exercise — 4



1. Briefly describe the design meaning of the above component diagram.
2. What is the role of the interface 'imageObserver' in the model ?

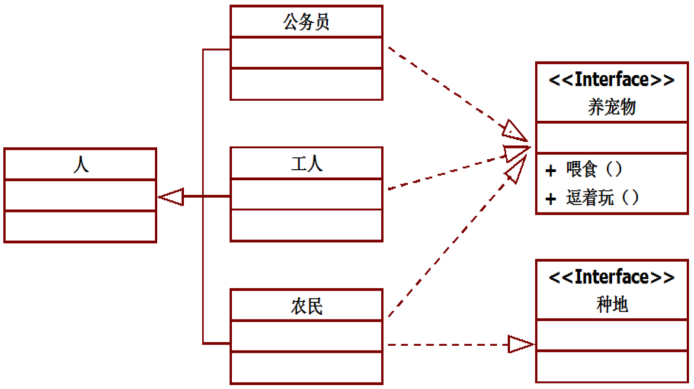
课堂作业一第四题 接口 构件图

类图示例 1



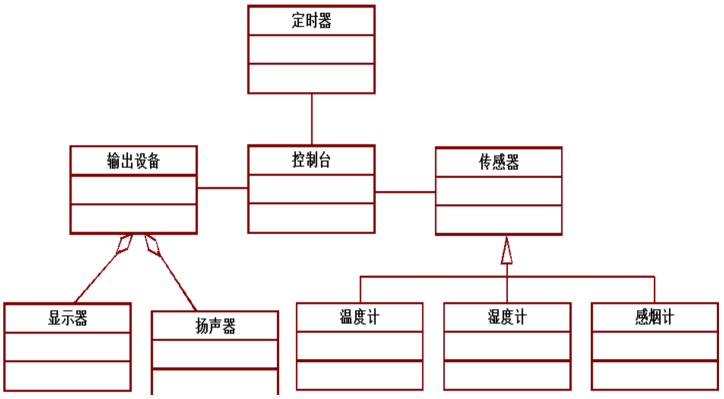
Player和曲撬是关联关系，Player和Team是聚合关系，前锋和Player是继承关系，球赛和冰球场是依赖关系

类图示例 2



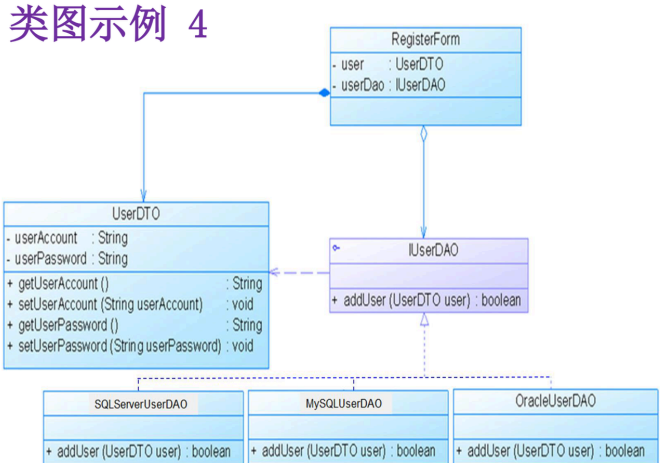
工人和人是继承关系，工人和养宠物是实现关系

类图示例 3



输出设备和显示器是聚合关系，传感器和温度计是继承关系，定时器和控制台是关联关系

类图示例 4



2.5 状态图P102、顺序图P100

📌 状态图 (State Machine Diagram)

状态图由五部分组成，分别是起点与终点、状态、转移、动作、以及复合状态。画图时，一般按照：识别所有核心状态、确定初始状态、梳理状态之间的转移条件、补充状态内部动作或全局终点的顺序画。

起点与终点

- **概念**：状态机的生命周期开始与结束的位置。一张状态图必须有且仅有一个起点，但可以有零个或多个终点。
- **画法**：
  - **起点（初始状态）**：一个**实心黑圆点**。
  - **终点（终止状态）**：一个**内部带实心黑圆点的圆圈**（同心圆）。

状态

- **概念**：对象在其生命周期中满足某种条件、执行某种活动或等待某个事件时的特定状况。
- **画法**：一个**圆角矩形**，里面写上状态的名称。
- **命名规范**：通常是**形容词、名词或被动动词**，如“未支付”、“处理中”、“已冻结”、“锁定状态”。

转移

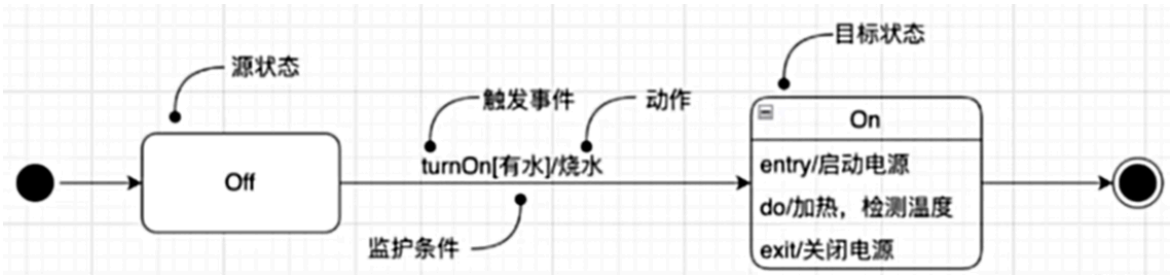
- **概念**：对象从一个状态变更为另一个状态的路径。通常由某个事件触发，并可能伴随特定的条件。
- **画法**：一条**带箭头的实线**，箭头由原状态指向新状态。
- **标准标注格式**：`事件名称 [守护条件] / 动作`（写在转移线上）。
  - **事件**：触发转移的信号（如 `输入密码`）。
  - **守护条件**：只有满足该条件时才允许转移，用中括号包围（如 `[密码错误次数 < 3]`）。
  - **动作**：转移时系统执行的临时操作（如 `错误计数加一`）。

动作

- **概念**：在状态内部执行的操作，不需要引发状态的转移。
- **画法**：在状态圆角矩形内部划一条横线，横线下方写上触发时机和对应动作。
- **常见触发时机关键字**：
  - `entry / 动作`：刚**进入**该状态时立即执行。
  - `exit / 动作`：准备**离开**该状态时立即执行。
  - `do / 动作`：驻留在该状态期间**持续执行**的活动。

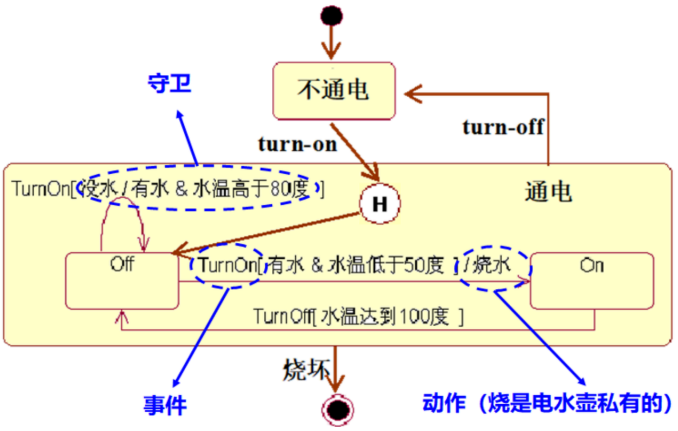
复合状态

- **概念**：包含一个或多个子状态机（内部还有完整的状态图）的状态，用于简化复杂的嵌套逻辑。
- **画法**：一个**嵌套的大圆角矩形**。大矩形代表父状态，其内部包含独立的起点、子状态、转移和终点。

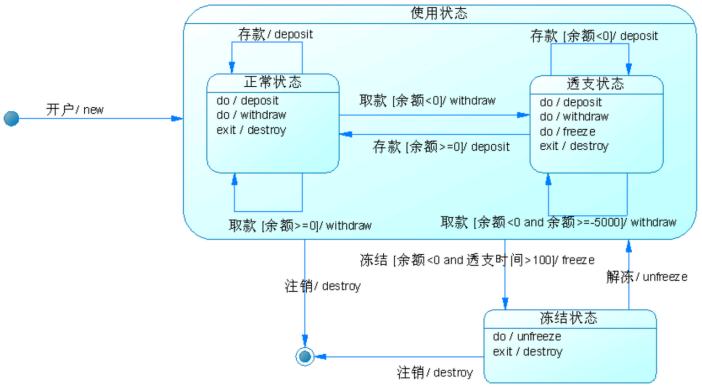


状态图示例

状态图示例 3



状态图示例 4



时序图 (Sequence Diagram)

顺序图由五部分组成，分别是对象、生命线、激活条、消息和销毁。画图时，一般按照：从左到右排列对象、从上到下绘制生命线、按时间顺序梳理并连线消息、补充激活条与控制流（如循环/分支）的顺序画。

对象 / 参与者

- **概念**：交互中所涉及的系统组件、类实例或外部用户角色。在图中通常按交互发生的先后顺序从左到右横向排列。
- **画法**：
  - **外部用户/系统**：使用小人（Actor）图标表示。
  - **类或对象实例**：使用**矩形框**表示。框内文字格式通常为 `对象名： 类名` （若不强调具体实例，可简写为 `:类名` ，并带下划线）。

生命线

- **概念**：表示对象在一段时期内的存在，时间轴沿着生命线**从上到下**流逝。

- **画法**：从对象/参与者矩形框向下延伸的一条**垂直虚线**。

激活条

- **概念**：表示对象正在执行某个操作、或者正在等待其他对象返回结果的时间段。
- **画法**：生命线上的**垂直窄矩形条**（覆盖在虚线之上）。当对象接收到消息并开始工作时，激活条开始；当操作结束或返回时，激活条结束。

消息

对象之间通过发送消息进行通信。消息依时间先后顺序从上到下排列。

同步消息

- **含义**：发送方发出消息后，**必须等待**接收方处理完毕并返回，在此期间发送方处于阻塞状态。
- **画法**：**实线 + 实心三角箭头**（`▶` 形状）。

异步消息

- **含义**：发送方发出消息后，**无需等待**接收方的响应，可以直接继续执行自己的后续操作。
- **画法**：**实线 + 开放式箭头**（`→` 或 `>` 形状）。

返回消息

- **含义**：接收方处理完同步消息后，向发送方返回结果或控制权。
- **画法**：**虚线 + 开放式箭头**。

自关联消息 / 自身调用

- **含义**：对象调用自身内部的方法或函数。
- **画法**：从自身的激活条向右拉出一条**折线（包含同步/异步箭头）**，重新指向自己下方的新嵌套激活条。

组合碎片

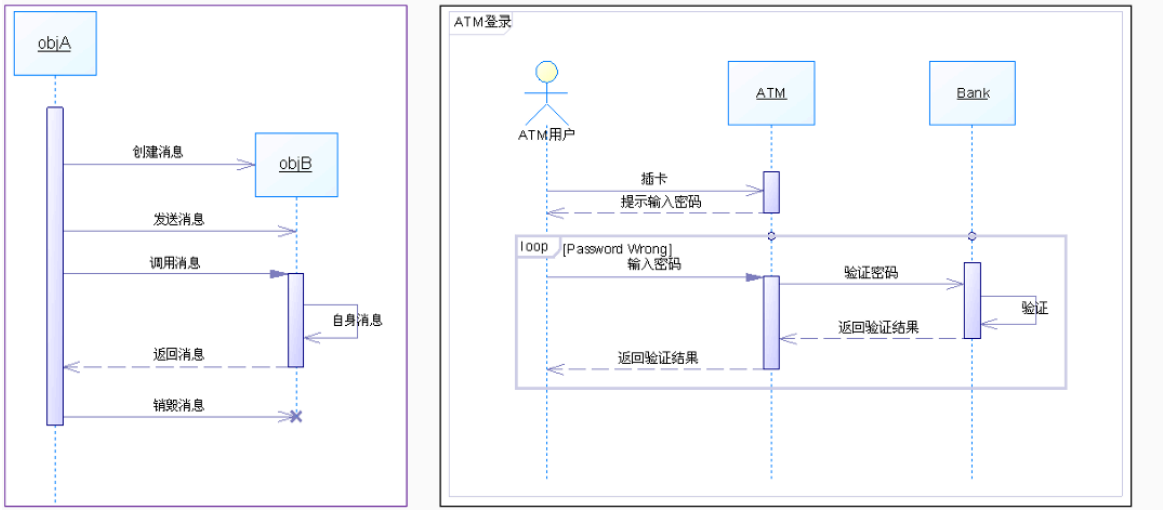
- **概念**：用于在顺序图内表达复杂的逻辑控制结构，如条件分支、循环等。
- **画法**：一个**透明的矩形大框**，包裹住受影响的消息。框的左上角有一个标签（操作符）表示逻辑类型。
- **常见操作符标签**：
  - `alt`（Alternatives）：**条件分支**，相当于 `if-else`。矩形内部用水平虚线隔开不同的条件分支区域。
  - `opt`（Option）：**可选分支**，相当于单分支 `if`，只有满足特定条件时才执行框内消息。
  - `loop`（Loop）：**循环结构**，标签旁可标注循环条件（如 `[i < 10]`），表示框内消息会重复执行。

Behavioral Modeling

时序图（Sequence Diagram）

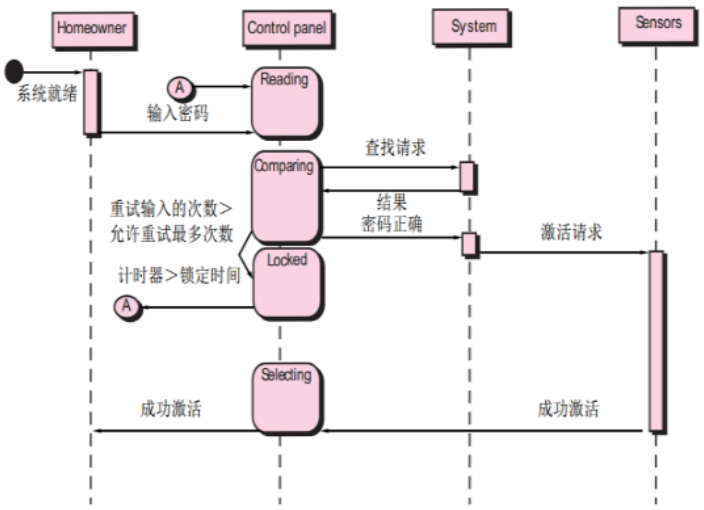
又称：顺序图或序列图，对应于行为视图

- 表示对象之间的交互，重点表示对象之间交互消息的时间顺序

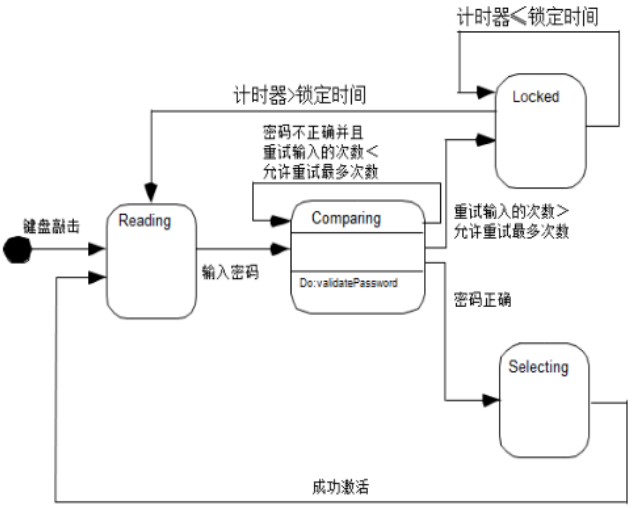


时序图示例

3、软件行为建模时，顺序图与状态图是常用的两种方法。



（a）



（b）

如图所示，在家庭安保 SafeHome 系统设计中，图（a）给出了系统安全功能的顺序图，图（b）给出了 ControlPanel 类的状态图，请解释两种建模方式的区别是什么 ？ （10 分）

21-22-2软工中文真题

2.6 基于场景建模

Scenario-Based Modeling

Why does **OO modeling** often **begin with** scenario-based modeling?

User satisfaction is the best way to measure the success of a computer-based system or product.

- modeling **begins with** identifying "**actor**".
- understand **how to interact with** a system, will be better able to properly characterize requirements and build meaningful analysis and design models.
- In essence, a use case **captures** the interactions that occur between producers and consumers of information within the system itself.

Which **UML diagrams** are useful in scenario-based modeling?

- Using UML to model requirements begins with the creation of scenarios in the form of **use case diagram**s, **activity diagram**s, and **sequence diagram**s.

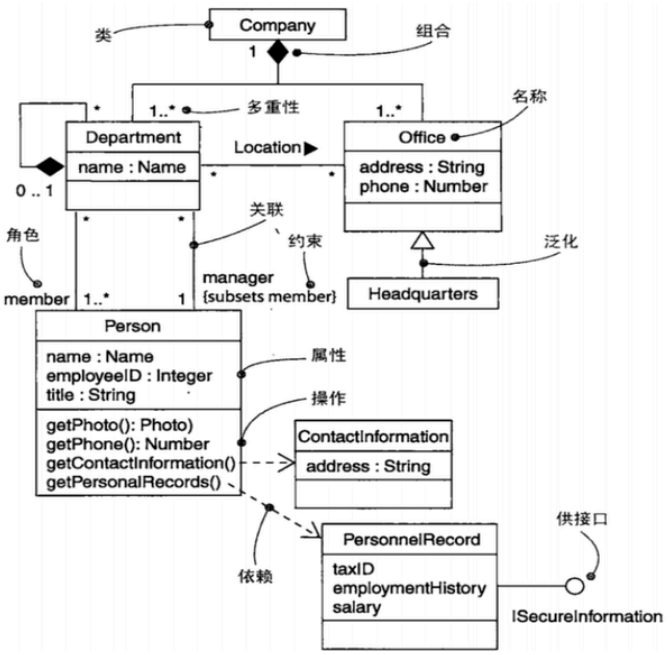
2.7 基于类建模 对象图

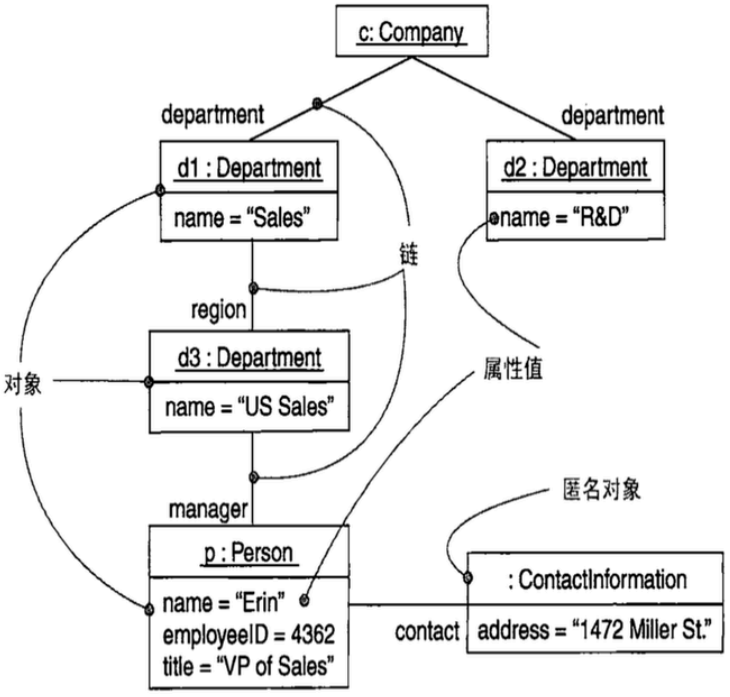
步骤：1. 识别分析类；2.定义属性与操作；3.CRC建模；4.绘制类图

对象图（Object Diagram）

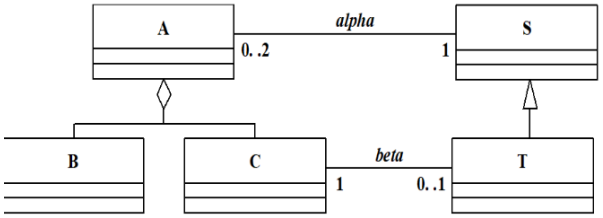
对应于结构视图

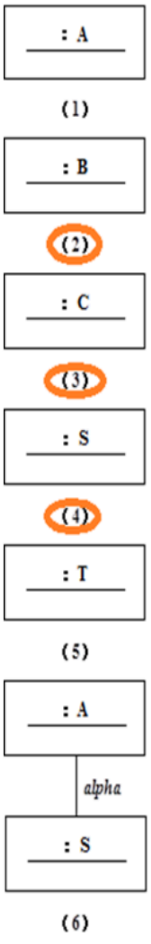
- 是类图在系统某一时刻时的静态快照，表示**类对象实例之间的链接关系**

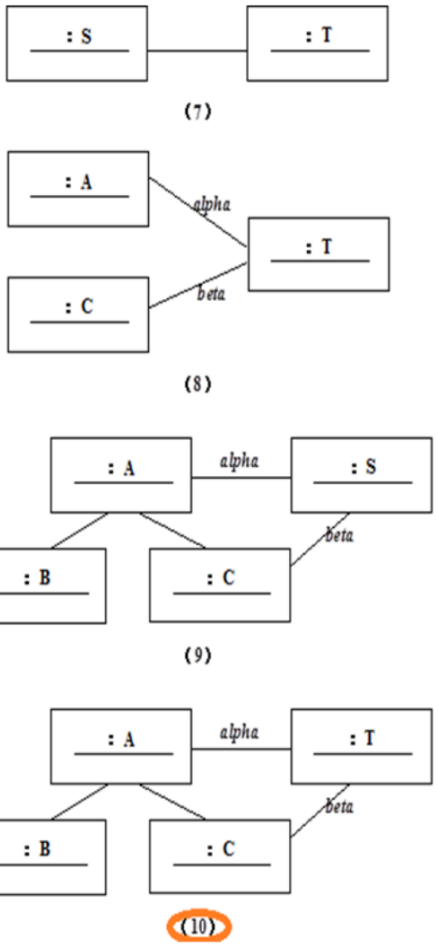




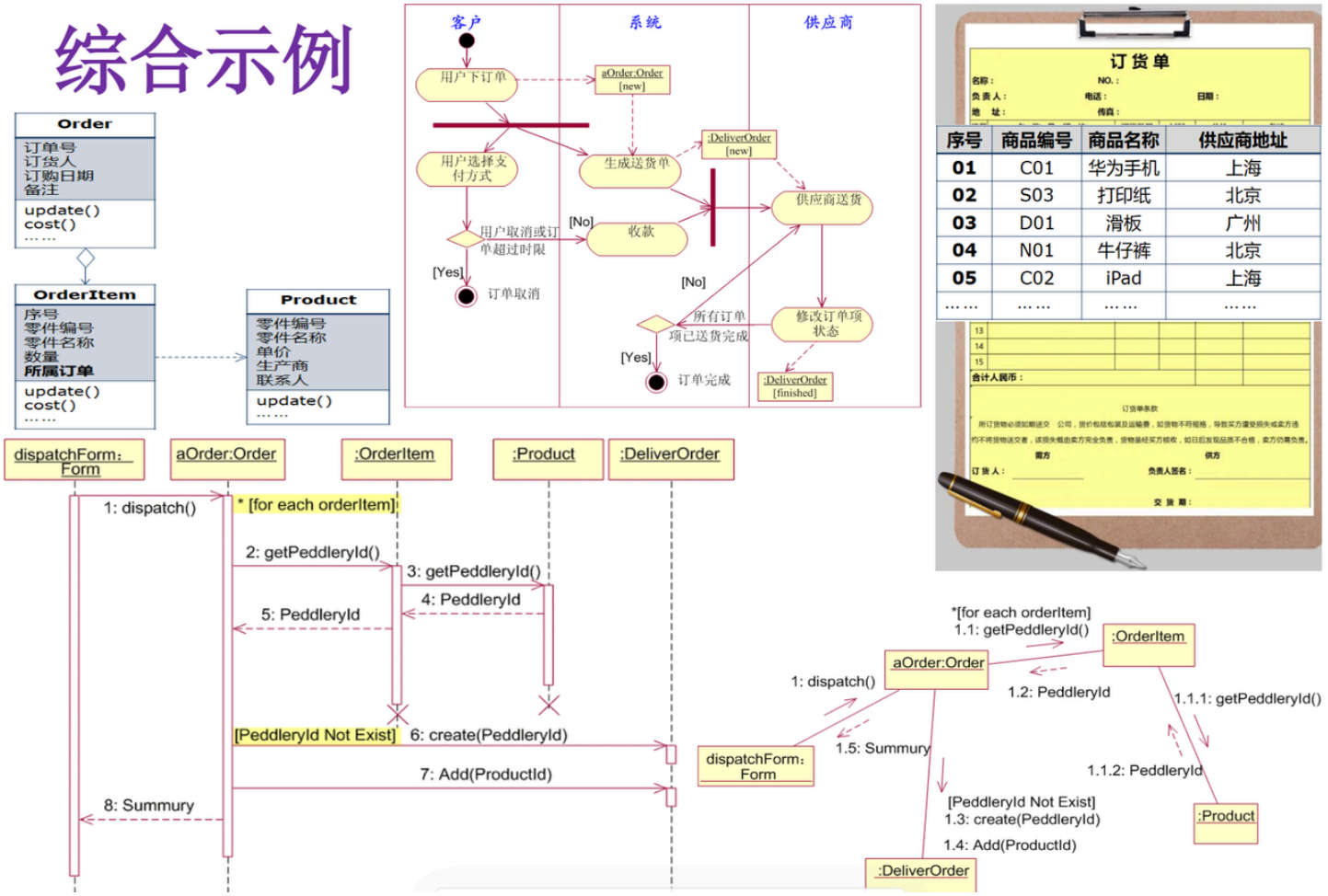
对象图示例







根据左侧类图判断右侧对象图哪些非法



左上角类图 中间上方活动图 左下角时序图 右下角通信图

3. 第三部分 设计和UI(第8，9，10，11章)

> 删除设计过程的指导原则内容

3.1 设计概念（Design Concepts）P145

设计概念（Design Concepts）

1. **抽象 (Abstraction)：** 在最高抽象水平，使用问题环境的术语描述解决方案；在较低水平，提供更详细的描述。
2. **体系结构 (Architecture)：** 软件的整体结构，包括模块的组织方式、交互方式以及数据结构。
3. **模式 (Patterns)：** 描述了一个在特定语境下解决常见设计问题的成熟方案。
4. **关注点分离 (Separation of Concerns)：** 建议将复杂问题分解为可以独立解决或优化的子部分。
5. **模块化 (Modularity)：** 软件被划分为独立命名且可寻址的模块，这是软件能够被人类理解和管理的单一属性。

6. **信息隐藏 (Information Hiding)：** 模块的设计应当使其内部包含的决策（如算法、数据结构）对其他模块不可见。
7. **功能独立 (Functional Independence)：** 开发具有“专一”功能且与其他模块过度交互较少的模块。
8. **逐步求精 (Stepwise Refinement)：** 一种自顶向下的设计策略，通过层层分解功能的细节，直到达到程序设计语言语句。
9. **重构 (Refactoring)：** 在不改变组件外部行为的情况下，简化设计或代码的组织技术，以提高内部结构的质量。
10. **设计类 (Design Classes)：** 将分析类进一步精化，提供更详细的实现细节（如属性、操作的完整描述）。

P149-152 功能独立性，独立性由内聚性和耦合性组成，设计目标应该是高内聚（Cohesion）和低耦合（Coupling）。内聚表示模块内部各个元素之间的关系，即模块内部元素各方法定义恰到好处；耦合是指模块与模块之间的关系，通过接口实现构建之间可控连接。

Cohesion:	**Coupling**:
• Definition: It indicates the relative functional strength of a module. • Key Points: It's a natural extension of the information-hiding concept. A cohesive module performs a single task, requiring little interaction with other components.	• Definition: It indicates the relative interdependence among modules. • Key Points: It refers to the interconnections among modules. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.

4.1 How effective modular design is achieved through functional independence of the individual modules?

中文翻译： 如何通过单个模块的功能独立性实现有效的模块化设计？

How effective modular design is achieved through functional independence of the individual modules?

Design Goal: **high cohesion** & **low coupling**.

independence is achieved by making modules single-minded and preventing excessive interaction with other system elements.

- design should ensure each module addresses a specific subset of requirements and has a simple interface.

independence is a key to good design, and design is the key to software quality. **Independent module** is

- easier to develop, because function can be compartmentalized and interfaces are simplified, and easier to perform parallel implementation of modules.
- easier to maintain & test, because secondary effects caused by modification are limited, error propagation is reduced, and reusable modules are possible.

课堂练习二4.1

4.2 What is the principle of information hiding as it applies to software design?

中文翻译： 应用于软件设计的信息隐藏原则是什么？

What is the principle of information hiding as it applies to software design?

- Principle suggests that modules should be " characterized by design decisions that (each) hides from all others ".
- Hiding implies that effective modularity can be **achieved** by defining a set of **independent modules** that communicate with one another only that information necessary to achieve software function.
- Abstraction helps to define entities (**procedural** or **informational**) that make up the software.
- Hiding defines & enforces access constraints to **both procedural detail** within a module **and** any **local data structure** used by the module.

课堂练习二4.2

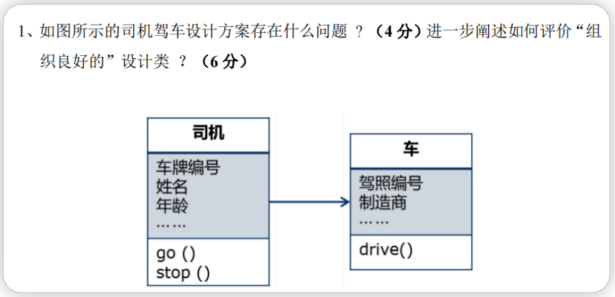
4.3 What is design refactoring?

中文翻译： 什么是设计重构？

What is design refactoring?

- a reorganization technique of changing a system in such a way that simplifies the design (or code) without altering **functionality and the external behavior** yet improves its **internal structure**.

课堂练习二4.3



21-22-2软工中文真题

设计类 P151 评估组织良好的类

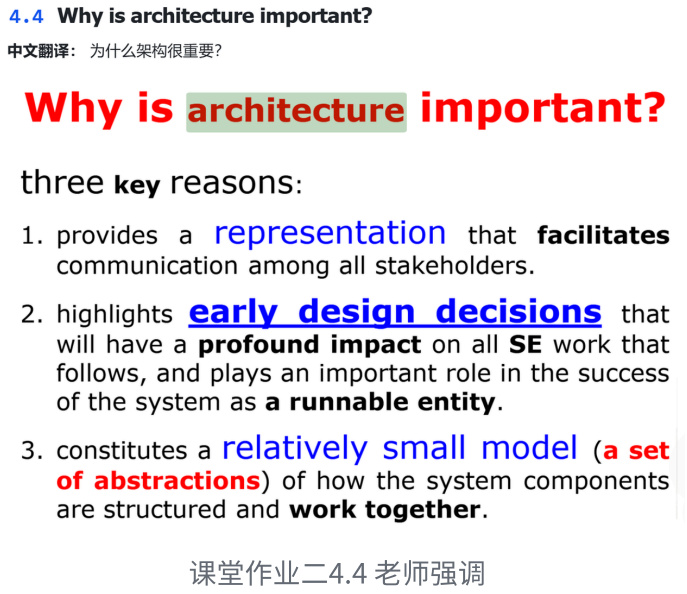
3.2 设计模型/设计模型元素 P153

📌 设计建模原则 P155黑字

1. **Principle 1: Design should be traceable to the requirements model.**
   设计应可追溯到需求模型
2. **Principle 2: Always consider the architecture of the system to be built.**
   始终考虑要构建系统的体系结构
3. **Principle 3: Design of data is as important as design of processing functions.**
   数据设计与处理功能设计同等重要
4. **Principle 4: Interfaces (both internal and external) must be designed with care.**
   接口（内部和外部）的设计必须谨慎
5. **Principle 5: User interface design should be tuned to the needs of the end user.**
   用户界面 design 应适应最终用户的需求
6. **Principle 6: Component-level design should be functionally independent.**
   构件级设计应在功能上独立

7. **Principle 7: Components should be loosely coupled to one another and to the external environment.**
   构件应彼此松耦合，并应与外部环境松耦合
8. **Principle 8: Design representations (models) should be easily understandable.**
   设计表示（模型）应易于理解
9. **Principle 9: The design should be developed iteratively.**
   设计应迭代地进行
10. **Principle 10: Creation of a design model does not preclude an agile approach.**
    设计模型的创建并不排除采用敏捷方法的可能性

3.3 体系结构设计（第九章）



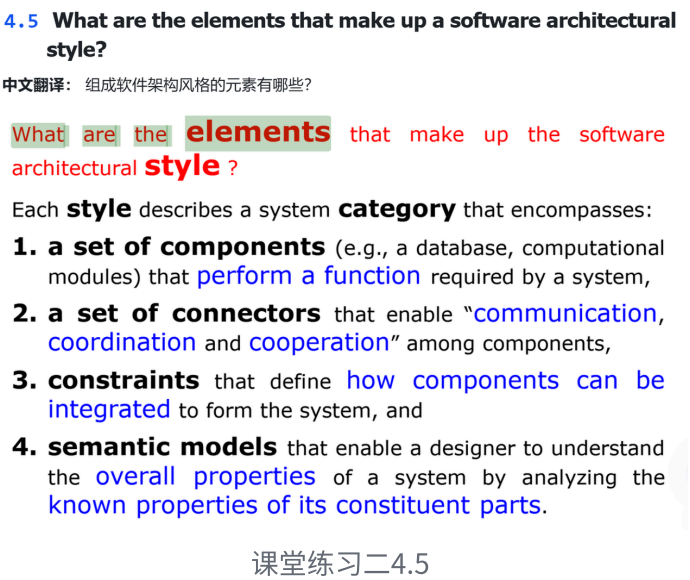
课堂作业二4.4 老师强调

定义：软件的整体结构

- Architecture is not operational software, it is a representation that enables a software engineer to:

1. Analyze the effectiveness of the design in meeting its stated requirements,
2. Consider architectural alternatives at a stage when making design changes is still relatively easy,
3. Reduce the risks associated with the construction of the software.

3.3.1 风格、模式 P169



课堂练习二4.5

风格实际描述一组通用的类别，是一个整体性描述

模式是局部范围内的，模式是一组抽象类，不是写好的代码

风格与模式的区别P169

风格由四个部分组成：一组构件、一组连接件、一组约束、一个语义模型

3.3.2 体系结构设计过程

4.6 What is an archetype?

中文翻译： 什么是原型（archetype）？

What is an archetype ?

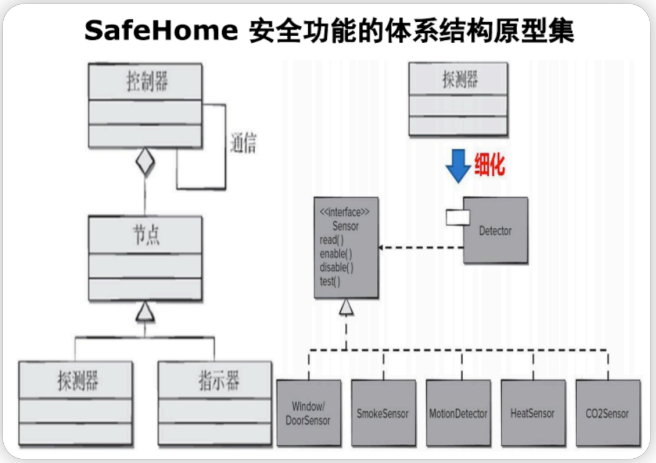
The target system architecture is composed of a **relatively small set** of **archetypes**.

Archetype design is **critical** to the design of an architecture.

An **archetype**

- is a **class** or **pattern** that represents **stable** element with a **core abstraction** for the target system.
- may be **instantiated iteratively** many **different ways** based on the **behavior** of the system.
- can be **derived** by examining the **analysis classes** defined as part of the requirements model.

课堂作业二题目六 老师强调



课本上的家庭安保系统这个模型实质上也是组合设计模式

3.4 构件级设计（第十章）

How does the **OO-View** of component-level design differ from the **traditional-view** ?

OO-View: a component contains a set of collaborating classes.

- A class is related to the problem domain and infrastructure domain.
- Each class within a component must be elaborated by defining the data structure of attributes, describing the process flow of operations.
- Specify the message details of all interfaces between the components.

Traditional-View: a module is a functional element of a program.

- three type of modules: control, business, infrastructure.
- All modules requires representations to be created for processing logic, the internal data structures, and an interface that enables the modules to be invoked and data to be passed to it.

A Process-Related View: CBSE

- assembling architecture by **reusing** components or design patterns.

课堂练习二5.1 面向对象的组件级设计视图与传统视图有何不同

3.4.1 基本设计原则 25分大题

🥇 **基本设计原则**

Principle 1: The Open-Closed Principle (OCP). 开闭原则

“一个模块（组件）应该对扩展开放，对修改关闭。”

Principle 2: The Liskov Substitution Principle (LSP). 里氏替换原则

“子类应该能够替换掉它们的基类（父类）。”

Principle 3: The Dependency Inversion Principle (DIP). 依赖倒置原则

“要依赖于抽象，不要依赖于具体实现。”

Principle 4: The Interface Segregation Principle (ISP). 接口分离原则

“多个特定于客户端的接口，要好于一个通用的综合接口。”

Principle 5: The Reuse/Release Equivalency Principle (REP). 发布/复用等价性原则

“重用的粒度就是发布的粒度。”

Principle 6: The Common Closure Principle (CCP). 共同封装原则

“一起修改的类应该被组合在一起。”

Principle 7: The Common Reuse Principle (CRP). 共同复用原则 P194-196

“不一起重用的类不应该被组合在一起。”

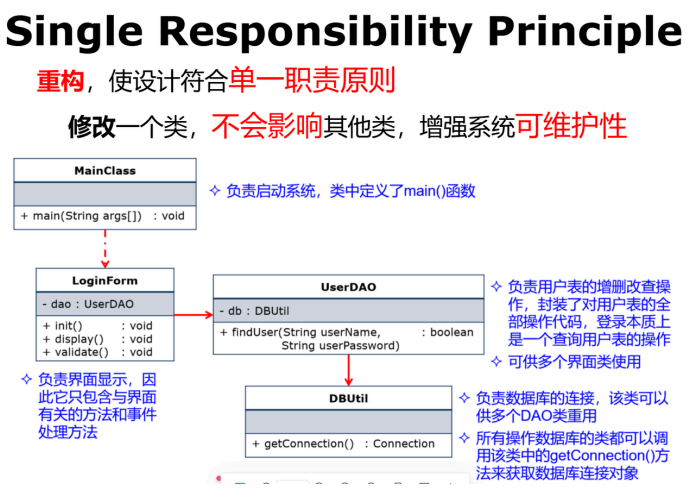
补充：

单一职责原则 (Single Responsibility Principle - SRP)

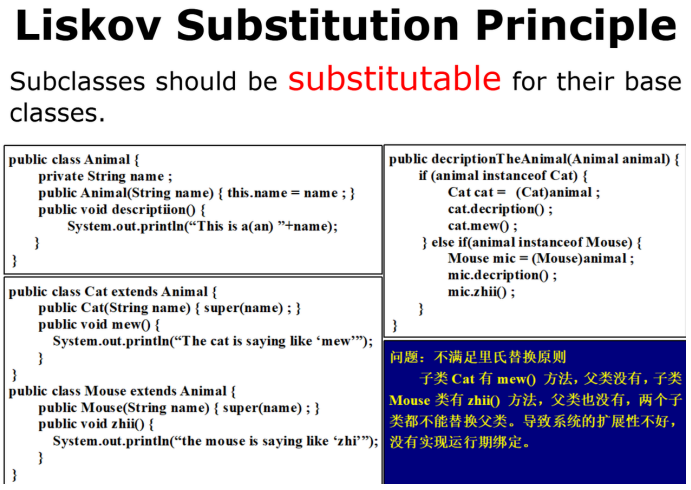
“每个类都应该只有一个职责，并且该职责应该完全被封装在这个类中。”

迪米特法则 / 最少知识原则 (Law of Demeter - LoD)

“每个单元对其他单元应该只有有限的了解：只了解与当前单元‘紧密’相关的单元。”



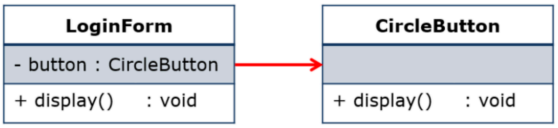
单一职责原则



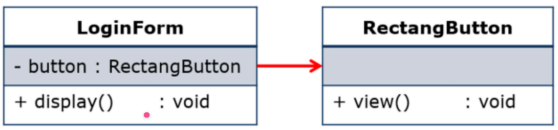
违反里氏替换原则示例

开闭原则OCP：示例

设计方案**违反**：开闭原则



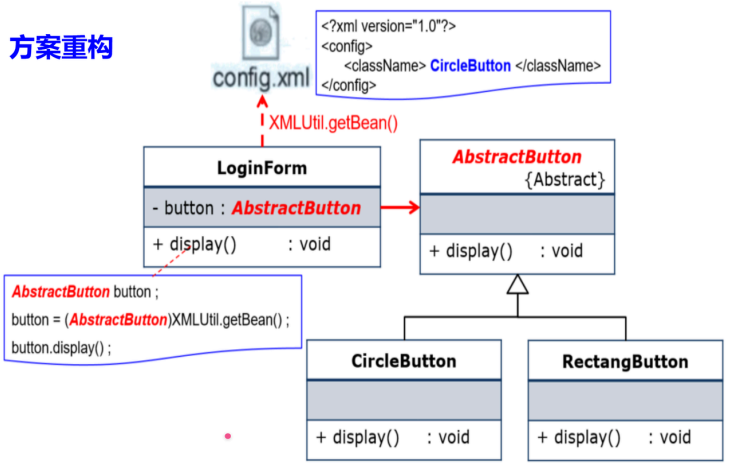
✧ 需求改变，需要修改 LoginForm 类的源码，修改按钮类的类名，因两个按钮类的显示方法不相同，需要修改 LoginForm 类的 display()方法



违反开闭原则示例

开闭原则OCP：示例

方案重构



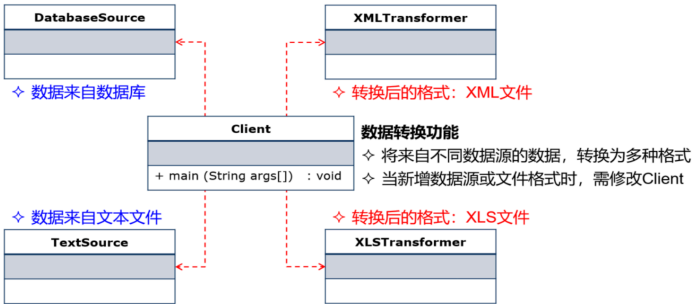
开闭原则修改示例

依赖倒置原则DIP：示例

以抽象方式耦合，是依赖倒置原则的关键

- LSP 是 DIP 的基础：抽象耦合关系一定涉及**具体类继承抽象类**，需保证在任何引用基类的地方，都可以替换成其子类

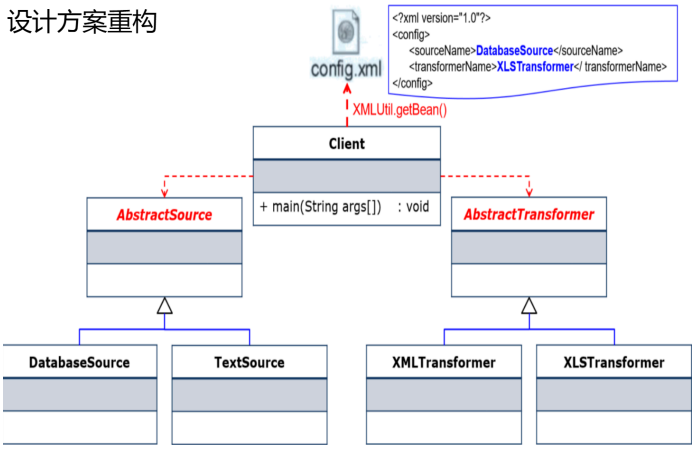
设计方案：违反开闭原则，**如何重构？**



违反依赖倒置原则示例

依赖倒置原则DIP：示例

设计方案重构

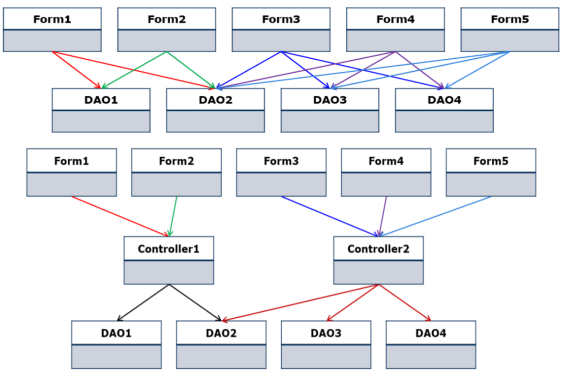


依赖倒置原则修改示例

LoD：example

界面类与数据访问类之间的调用关系复杂，耦合度大

- 新增界面或新增数据访问麻烦，系统可维护性、扩展性差



设计重构

最少知识原则

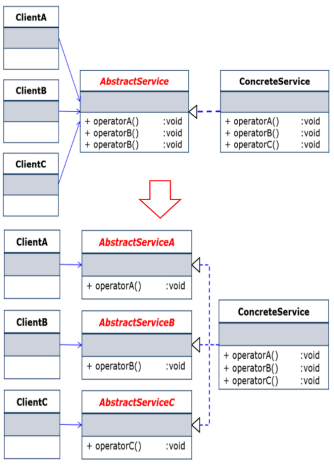
Interface Segregation Principle

Many client-specific interfaces are better than one general purpose interface.

- The client (**design class**) **does not rely on** unnecessary interfaces.
- **Just need to know** the relevant methods (a controlled connection).

ISP suggest:

- The operations of the design class are categorized into one or more abstract classes (or interface).
- operations within the abstract class (the interface) should be cohesive; that is, it should exhibit a limited function.



接口隔离原则

接口分离原则ISP

设计是否已经最优？

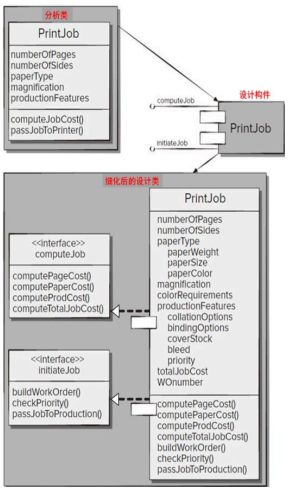
构件 PrintJob 与接口 initiateJob 应重构：

- initiateJob 接口**分解**为 3 个接口：buildJob、submitJob、initiateJob，每个接口仅关注一个功能
- 构件 PrintJob **新增**3个类：

  WorkOrder类中操作 buildWorkOrder()负责组装一个工作单；

  JobQueue类中操作 checkPriority()检查任务优先级；

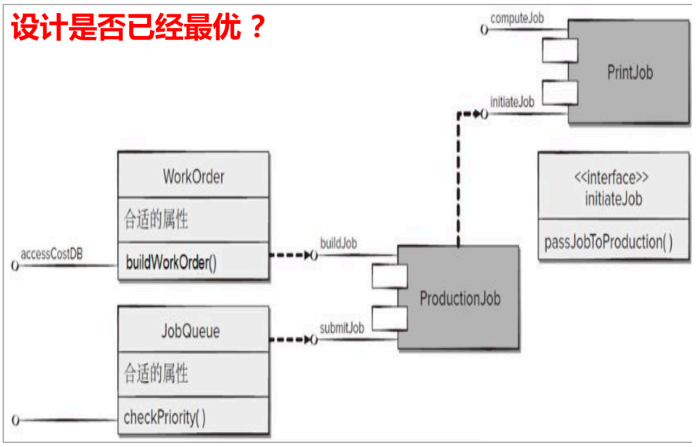
  ProductionJob类，负责将任务传递到生产设施，进行生产作业



接口分离原则更改前

接口分离原则ISP

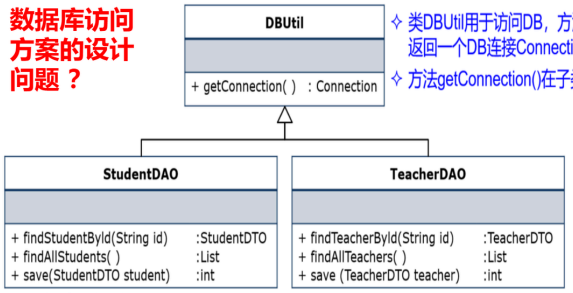
设计是否已经最优？



接口分离原则更改后

共同复用原则CRP：示例

数据库访问方案的设计问题？



✧ 类DBUtil用于访问DB，方法getConnection()返回一个DB连接Connection的对象

✧ 方法getConnection()在子类中被**继承复用**

设计方案的问题：

✧ DB支持多种连接方式：1) JDBC连接；2) 池连接类

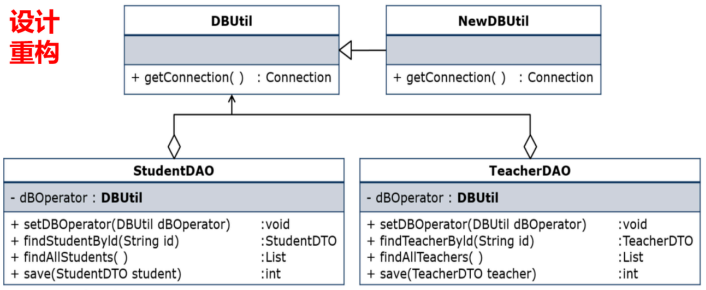
✧ 若两个子类采用不同连接，需新增一个DBUtil类，且修改StudentDAO类或TeacherDAO类，继承新的DBUtil类

✧ 违反开闭原则，系统扩展差

共同复用原则更改前

共同复用原则CRP：示例

设计重构



设计方案改进：

✧ StudentDAO类、TeacherDAO类与DBUtil类不再是继承关系，改为聚合关联关系

✧ 通过setDBOperator()函数，为类的成员变量dBOperator赋值

✧ 更换连接方式时，1）新增一个DBUtil类的子类，2）子类重写getConnection()函数，3）客户端调用setDBOperator()函数时，注入子类对象即可

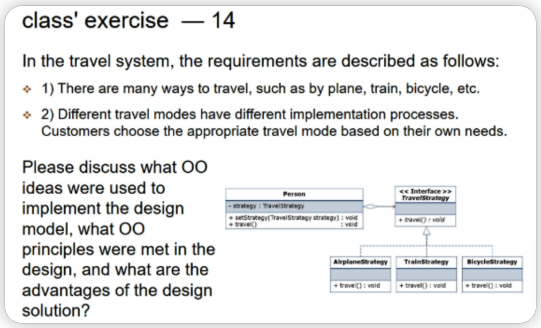
✧ 客户端只针对DBUtil类编程，符合开闭原则

共同复用原则更改后

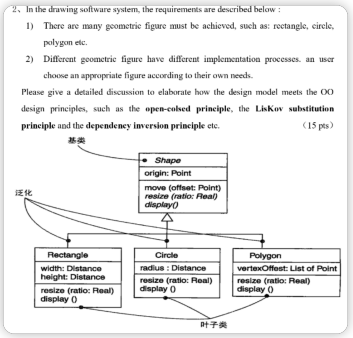
对于这类题目常见的问题是，给出一个类图，问题：针对这样一个问题的设计方案，运用了什么样的设计原则，体现了什么样的设计概念，它的好处是什么？

1. 阐述面向对象的**多态**概念 -- OO的概念
2. 这样的设计方案，运用了里氏替换原则和依赖倒置原则，解释一下这两个原则，从而实现了开闭原则，做到了对外延具有开放性，对修改具有封闭性；
3. 好处：可扩展性和可维护性

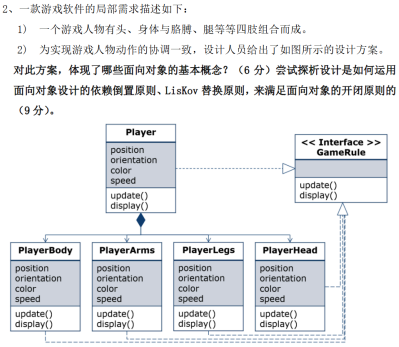
> 典型例题三移至课堂作业一最后（接口隔离原则）



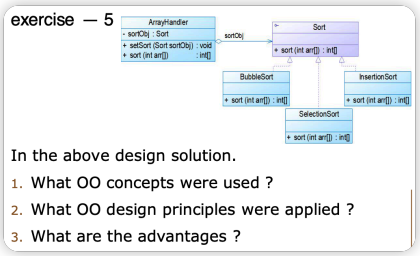
典型例题一 通过接口实现的多态



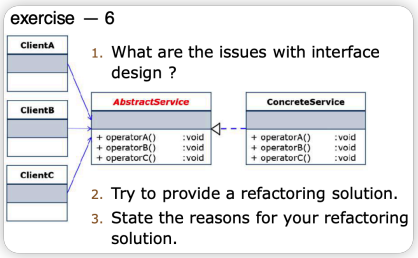
21-22-2双语真题 典型例题二 通过继承实现的多态



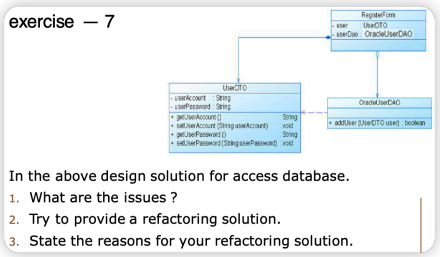
21-22-2 典型例题四 中文真题



课堂作业一题目五



课堂作业一题目六



课堂作业一题目七

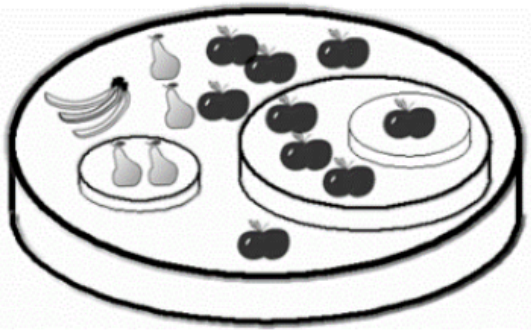
3.4.2 构件级设计过程/步骤

实施构件级设计（Conducting Component-Level Design）P201开始小标题

What are the **steps** used to complete the **component-level** design for a software development project?

1. Identify **design** classes that correspond to **problem** domain.
2. Identify **design** classes that correspond to **infrastructure** domain.
3. Elaborate all **design** classes that are **not** acquired as **reusable** components. 细化构成所有非可复用构件的设计类
4. Identify persistent **data** sources and identify the **classes** required to **manage** them.
5. Develop and elaborate **behavioral representations** for each class or component.
6. Elaborate **deployment** diagrams to provide **additional** implementation **detail**.
7. Refactor every **component-level representation** and always consider **alternatives**.

2、The combination of fruit plates is shown in the figure. How will you design and develop this system? Please use the class diagram to give the design model and explain the reasons for choosing the scheme.? （10 pts）



21-22-2双语真题

3.5 用户体验设计（第11章）

1. 什么是用户体验设计 (UX Design)？P215

- **定义：** 用户体验设计是通过在产品及其用户之间建立一种**可用、可访问且令人愉悦**的交互，来增强用户满意度的过程。
- **核心：** 成功的软件产品必须提供积极的用户体验。

2. **为什么从 UI 转向 UX（追求什么）？ P216 P157**

- **范畴更广：** UX 设计比传统的用户界面设计、可用性工程或可访问性工程的范围更广。
- **追求目标：** 工程师不再仅仅关注界面看起来如何，而是追求**确保用户界面设计的“可用性”(Usability)**。
- **全过程关注：** UX 设计要求从开发过程的每一步就开始考虑用户的动作和期望，使打造积极用户体验的任务变得更具可控性。

3. **可访问性与可用性 (Accessibility & Usability) P215 P218 P219**

- **可用性 (Usability)：** 衡量用户使用高科技产品所提供的功能和特性的**简便程度和效率**的定性指标。可用性工程旨在设计出易于学习、易于使用且易于记忆的界面。
- **可访问性 (Accessibility)：** 指系统为**具有特殊需求的人群**（如视力障碍、听力障碍、老年人或认知障碍者）提供感知、理解、导航和与产品交互的手段的程度。UX 设计必须将这些辅助技术考虑在内。

4. **黄金法则 P220开头**

- 置用户于控制之下 (Place the user in control)
- 减少用户的记忆负担 (Reduce the user’ s memory load)
- 保持界面一致 (Make the interface consistent)

4. 第四部分 软件测试（第15，16章）

验证和确认P285 课堂练习二6.1

验证是保证正确实现特定功能（正确的构建产品）

确认是保证满足客户需求（构建正确的产品）

软件测试是为了**发现**在设计和构造过程中无意中引入的**错误。(不是验证正确性)P308**

单元测试（unit testing）----->集成测试(integration testing)----->确认测试(validation testing)----->系统测试(system testing)

4.1 构件测试

4.1.1 传统软件的测试策略

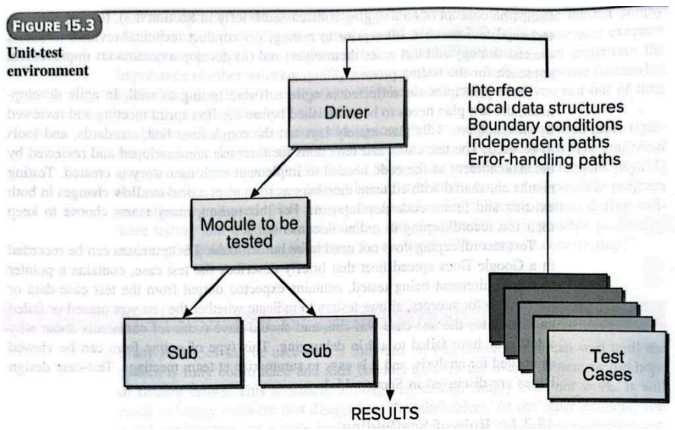
view: OO & traditional

How does the **OO-View** of component-level design differ from the **traditional-view**?

- OO-View: a component contains a set of collaborating classes.
  A class is related to the problem domain and infrastructure domain.
  Classes are elaborated by defining all attribute data structures, describing process flow for operations, specifying messaging details for interfaces.
- Traditional-View: a module is a functional element of a program.
  three type of modules: control, business, infrastructure.
  All modules requires representations to be created for processing logic, the internal data structures, and an interface that enables the modules to be invoked and data to be passed to it.
- **A Process-Related View**: **CBSE**, assembling architecture by reusing components or design patterns.

PPT：OO-View和传统观点的区别

4.1.2 测试用例设计



P292单元测试环境图 待测模块，必须有driver驱动，sub（stub桩）

驱动程序就是主程序，是为了测试自己写的模块，让模块运行起来。

桩是指替换真正的代码，只提供我们的模块测试所需的数据即可，因为目前讨论的是构建级别的测试，每个程序员只完成自己的任务，但是可能会调用别人的代码，这时只需要那部分代码运行的结果所产生的数据，即用桩进行替代。

4.1.3 白盒测试 P295

What is the difference between black-box testing and white-box testing?

Black-box testing involves testing the functionality of a software component without knowing the details of its internal logic.

White-box testing involves testing the independent logic paths with full implementation knowledge.

课堂练习二第六章第二题 白盒黑盒测试的区别

利用白盒测试方法导出的测试用例 可以：

1. 保证一个模块中的所有独立路径至少被执行一次；

White Box Testing

Using white-box testing methods, you can derive test cases that:

1. Guarantee that all independent paths within a module have been exercised at least once.
2. Exercise all logical decisions on their true and false sides.
3. Execute all loops at their boundaries and within their operational bounds.
4. Exercise internal data structures to ensure their validity.

PPT：利用白盒测试方法导出的测试用例

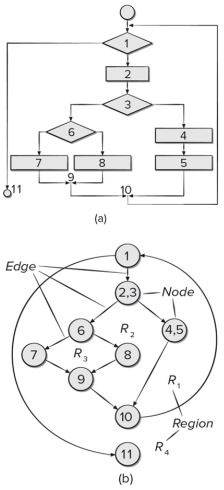
2. 对所有的逻辑判定均需测 试取真 (true) 和取假 (false) 两个方面；
3. 在上下边界及可操作的范围内执行所有循环；
4. 检验内部数据结构以确保其有效性

4.1.3.1 基本路径测试 (复习PPT上标红，考大题) 流图（程序图） P296-297

Basis Path Testing

Determine the number of independent paths in the program by computing **Cyclomatic Complexity**:

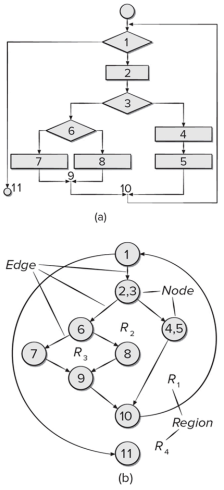
1. The number of **region**s of the flow graph corresponds to the cyclomatic complexity.
2. Cyclomatic complexity $V(G)$ for a flow graph G is defined as：
   $$V(G) = E - N + 2$$
   E is the number of flow graph **edge**s
   N is the number of flow graph **node**s.
3. Cyclomatic complexity $V(G)$ for a flow graph G is also defined as:
   $$V(G) = P + 1$$
   P is number of predicate **node**s contained in the flow graph G.



PPT1：如何计算环复杂度

example

1. Flowchart ⇒ Flow Graph
2. compute Cyclomatic Complexity
   - The flow graph has 4 regions.
   - V(G)=11(edges) − 9(nodes) + 2 = 4
   - V(G)=3(predicate nodes) + 1 = 4
3. An independent path is any path through the program that introduces at least one new set of processing statements or a new condition (need 4 independent paths to test)
   - path 1: 1-11
   - path 2: 1-2-3-4-5-10-1-11
   - path 3: 1-2-3-6-8-9-10-1-11
   - path 4: 1-2-3-6-7-9-10-1-11



PPT2：基本路径测试的例题

第一个问题是将流程图转化为流图，只要将顺序执行的结点合并即可

第二个问题是求环路复杂度，一共有三种求解方法：

1. 求封闭区域的个数，复杂度为封闭区域个数+1，如上图中R1,R2,R3都为封闭区域
2. 流图中 $V(edges) - V(nodes) + 2$
3. 流程图中分支点的数量（即菱形数量）+1

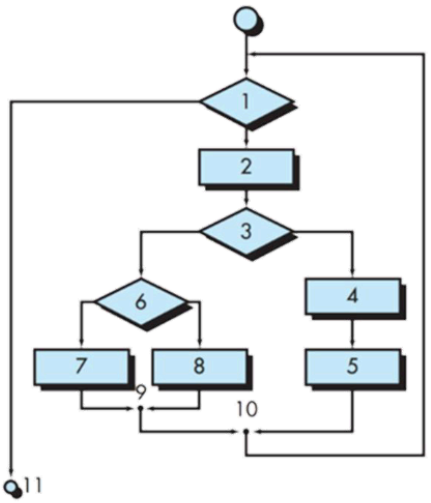
第三个问题是求所有的独立路径，独立路径的数量就是环路复杂度，要保证每一条分支路径都被走一遍，且一定要从起点走到终点，如上图中是从1到11

4、As shown in the figure, a flowchart is used to depict program control structure.

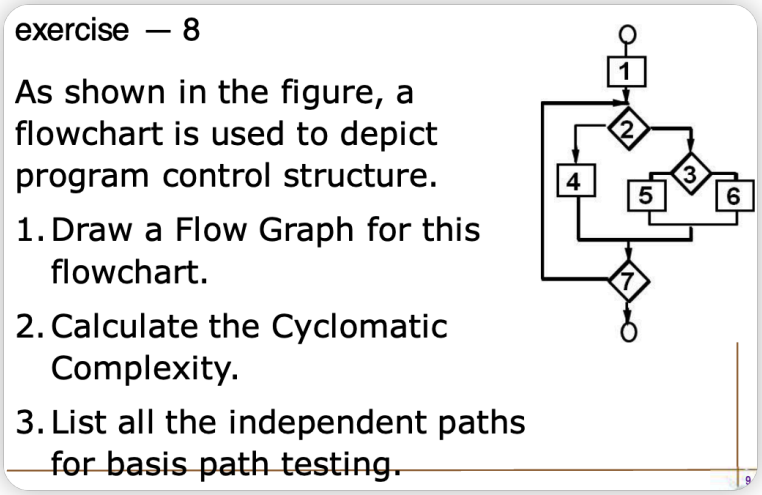
1) Draw the corresponding flow graph using the number in the flowchart.（4 pts）

2) Calculate the cyclomatic complexity of the procedure.（2 pts）

3) List all the independent paths for basis path testing.（4 pts）



21-22-2双语真题



课堂作业一题目八

4、如图示，给了一段计算输入数据绝对值的程序代码。

1) 请使用代码前面的序号，绘制流图（4 分）

2) 计算程序的环路复杂性（2 分）

3) 列出基本路径测试的独立路径（4 分）

```
    begin
①   sum=0;
    input a;
②   do while a!=0 {
③           if (a>0)
④                   { sum+=a; }
            else
⑤                   { sum-=a; }
⑥           input a;
    }
⑦   if(sum>0)
⑧           printf("The sum of your input is: %d",sum);
    else
⑨           printf("The sum of your input is: -999");
⑩   end
```

21-22-2软工中文真题

4.1.4 黑盒测试 P300

Black Box Testing

Black-box (functional) testing attempts to find errors in the following categories:

- Incorrect or missing functions.
- Interface errors.
- Errors in data structures or external database access.
- Behavior or performance errors.
- Initialization and termination errors.

Unlike white-box testing, which is performed early in the testing process, **black-box testing** tends to be applied during later stages of testing.

PPT：黑盒测试是要发现什么类别的错误

Black Box Testing

Black-box test cases are created to answer questions like:

- How is functional validity tested?
- How are system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?
- What effect will specific combinations of data have on system operation?

PPT：黑盒测试回答什么问题

4.1.4.1 等价类划分

black-box: Equivalence Partitioning

What is equivalence partitioning in the black-testing?

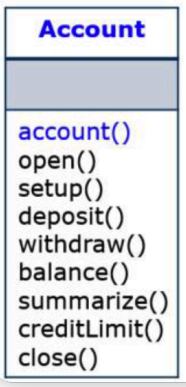
- Black-box testing technique in which the input domain is divided into **a valid** equivalence class and **some invalid** equivalence classes.
- Test cases are derived from combinations of elements from each equivalence class.
- Exhaustive testing of all input domain values is not necessary.

guidelines: If an input condition

- specifies a **range**: one **valid** & two **invalid**
- requires a **specific value**: one **valid** & two **invalid**
- specifies **a member of a set**: one **valid** & one **invalid**
- is **Boolean**: one **valid** & one **invalid**

PPT：什么是等价类划分

exercise — 9



❖How to test the **Account** class based on the method of equivalence partitioning ?

课堂作业一题目九

4.1.4.2 边界值分析 BVA

black-box: Boundary Value Analysis

BVA leads to a selection of test cases that exercise bounding values.

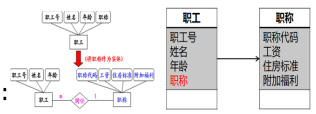
Guidelines for BVA:

1. If an input condition specifies a range bounded by values a and b, test cases should be designed with values a and b and just above and just below a and b.
2. If an input condition specifies a number of values, test cases should be developed that exercise the min and max numbers as well as values just above and below min and max.
3. Apply guidelines 1 and 2 to output conditions.
4. If internal program data structures have prescribed boundaries (for example, array with max index of 100) be certain to design a test case to exercise the data structure at its boundary.

PPT：边界值分析

4.1.5 OO测试

OO Testing



3 things to test OO systems:

1. The definition of testing must be broadened to include error discovery techniques applied to OO analysis and design models.

   早期技术评审发现语法、语义和语用方面的错误，评审模型正确性、完整性和一致性，可防止错误向后期蔓延

2. The strategy for unit and integration testing must change significantly.

   OO类是测试的重点，类中的操作（方法）是最小可测试单元

```
class student { …… }
student t = new student()
……
t.study()
……
```

3. The design of test cases must account for the unique characteristics of OO software.

   OO环境中独立测试一个操作是无效的；由于多态，不同子类因有不同属性与不同操作逻辑，其上下文环境不同于其他子类

PPT：构件级OO测试要做的3件事

What is object-oriented unit testing?

OO **class** testing is equivalent to **unit** testing of conventional software.

- Conventional unit testing focus on the **algorithmic detail** of a module and the **data** that flow across the module interface.
- OO class testing is driven by the **operations** encapsulated by the class and the **state behavior** of the class.
- The focus throughout is on **designing and testing appropriate sequences of operations** to **exercise all class states**.
- It makes little sense to **test operations or algorithms individually** for classes.

课堂练习二6.3 什么是面向对象的单元测试 课上强调

4.2 集成测试

4.2.1 白盒测试&黑盒测试

PPT：任何产品都可通过2种方式来测试，黑盒是外部视角、白盒是内部视角：

Approaches to Testing

Any engineered product can be tested in one of two ways:

1. External Views (Black-box testing)

- Knowing the specified function that a product has been designed to perform,
- tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function.

2. Internal Views (White-box testing)

- Knowing the internal workings of a product,
- tests can be conducted to ensure "all gears mesh", that is, internal operations are performed according to specification and all internal components have been adequately exercised.

4.2.2 集成测试（上-下、下-上）

自顶向下集成 (Top-Down Integration) P310-311

Top-Down Integration

an **incremental** approach to **construction** of the **architecture**

- verifies major **control** or **decision** points early in the test process.

How does depth-first differ from breadth-first when using a top-down integration strategy?

- **depth-first**: integrates all components on a major control path of the program structure.
- **breadth-first**: incorporates all components directly subordinate at each level, moving across the structure horizontally.
- If **depth-first** integration is selected, **a complete function** of the software **may be implemented** and demonstrated. Early demonstration of functional capability is **a confidence builder** for all stakeholders.

- **定义：** 这是一种增量式的构造软件体系结构的方法。模块按照控制层次结构，从主控制模块（主程序）开始，逐层向下进行集成。
- **集成方式：**
  - **深度优先 (Depth-first)：** 沿着主控制路径向下集成所有的模块。
  - **宽度优先 (Breadth-first)：** 按照水平方向，逐层遍历所有的直接下级模块。

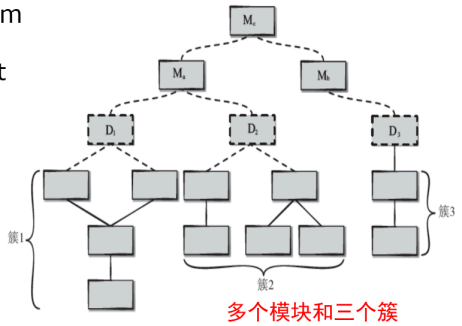
- **核心工具——存根 (Stubs)：** 在测试高层模块时，下层模块尚未开发或集成，因此需要编写“存根”程序。**存根是模拟下层模块功能的临时替代品**，用于接收高层模块的调用。
- **执行步骤：**
  a. 用主控制模块作为测试驱动器，所有直接下属模块用存根代替。
  b. 根据选择的集成策略（深度或宽度优先），每次替换一个存根模块。
  c. 在集成每个模块时进行测试。
  d. 完成一套测试后，用真实的模块替换下一个存根。
  e. 进行**回归测试**，确保没有引入新错误。

自底向上集成 (Bottom-Up Integration) P311-312

Bottom-Up Integration Testing

begins construction and testing with atomic modules components at the lowest levels in the program structure.

1. Low-level components are combined into clusters (builds) that perform a specific software subfunction.
2. A driver (a control program for testing) is written to coordinate test-case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined, moving upward in the program structure.

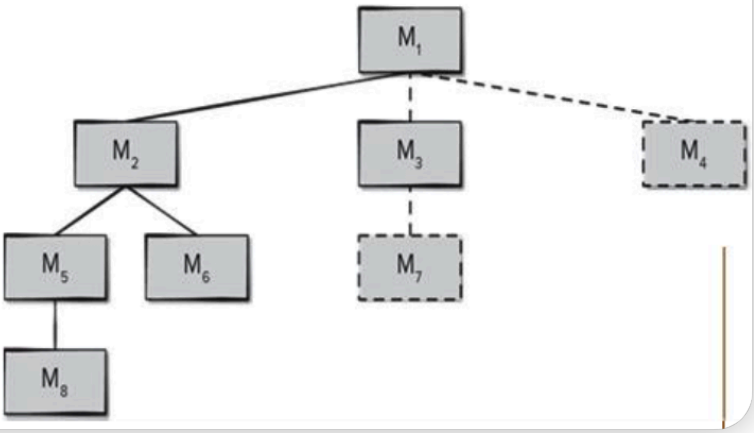


多个模块和三个簇

- **核心工具——驱动器 (Drivers)：** 因为是从底层开始，高层控制模块尚未就绪，所以需要编写“驱动器”。**驱动器是用于控制测试输入和输出的程序**，模拟高层模块对底层模块的调用。
- **执行步骤：**
  a. 将低层模块组合成实现特定软件子功能的**簇（Clusters，有时称为构建）**。
  b. 编写驱动器来协调测试用例的输入和输出。
  c. 对簇进行测试。
  d. 去掉驱动器，将簇合并，沿着程序结构向上移动。

exercise — 10

❖ Why is a depth-first strategy used in the top-down integration testing for large software systems ?



课堂作业一题目十 需要写出深度优先遍历序列

4.2.3 OO测试 P302

OO Integration Testing

Goal of testing has not changed, but the strategy and technique has changed due to the OO features of software.

- the system does not have an obvious hierarchical control structure
- direct and indirect interactions of the components that make up the class.

two different strategies

1. **Thread-based testing**, integrates the set of classes required to respond to one input or event for the system.
   Each thread is integrated and tested individually. Regression testing is applied to ensure no side effects occur.
2. **Use-based testing**, begins the construction of the system by testing **independent classes** that use very few server classes.
   The next layer **dependent classes** use the independent classes are tested next. This **sequence of testing layers of dependent classes continues** until the entire system is constructed.

集成级OO测试的两种不同的策略：

1. 基于线程的测试
2. 基于使用的测试

5. 第五部分 配置管理（第17章）

1. 软件配置管理管什么 P324 划重点

程序数据文档，产生的工件配置项

2. 软件配置管理与软件支持的区别 P324上半部分第二段 划重点

- **软件支持 (Software Support)：** 是指在软件交付给客户并投入运行之后发生的一系列活动，侧重于纠错、环境适应和功能增强。

- **软件配置管理 (SCM)：** 是一组**跟踪和控制活动**。它从软件工程**项目开始时即启动**，直到软件**退役（停止运行）时才结束**。
- **对照关系：** SCM 贯穿整个软件生命周期，而支持仅发生在交付后。SCM 为有效的支持提供了基础保障。

What information will make up the software configuration?

- Software configuration are work products that created by engineers during the software process.

  computer programs (source code and executables), documentation (technical and user), data (internal and external to programs)

课堂练习二7.1 哪些信息构成软件配置

5.1 变更

5.1.1 核心定义与目的

变更控制是一种在软件工程项目中将**人为制定的规程与自动工具结合起来**的规范化管理机制。其根本目的是为了防止大型项目中“不受控制的变更迅速导致混乱”，确保变更在可控、可追溯的条件下进行。

What are the **steps** in the change control process?

1. **evaluate** a change request, and **submit** a change report to **CCA** (change control authority). The **CCA** makes the final determination as to the **status** and **priority** of the change.
2. An **ECO** (engineering change order) is generated for each **approved change**.
3. Change SCIs are **checked out** from the project database subject by its access control **parameters**.
4. The **modified SCIs** is subjected to **SQA** procedures and **checked into** the project database.
5. **Version control** procedures are followed to produce the **next version** of the software. Adopting **Synchronization control** ensures that **parallel changes** made by different people do **not overwrite** with each other.

课堂练习二7.2 项目变更控制过程的步骤是什么

5.1.2 变更管理的两个主要元素

变更控制过程与版本控制机制集成在一起，共同实现了变更管理的两个核心元素：

- **访问控制（Access Control）**：负责管理哪个软件工程师有权限访问和修改某个特定的配置对象（SCI）。
- **同步控制（Synchronization Control）**：协助保证两个不同的人员完成的并行变更不会被相互覆盖。

5.1.3 变更控制的三个实施阶段

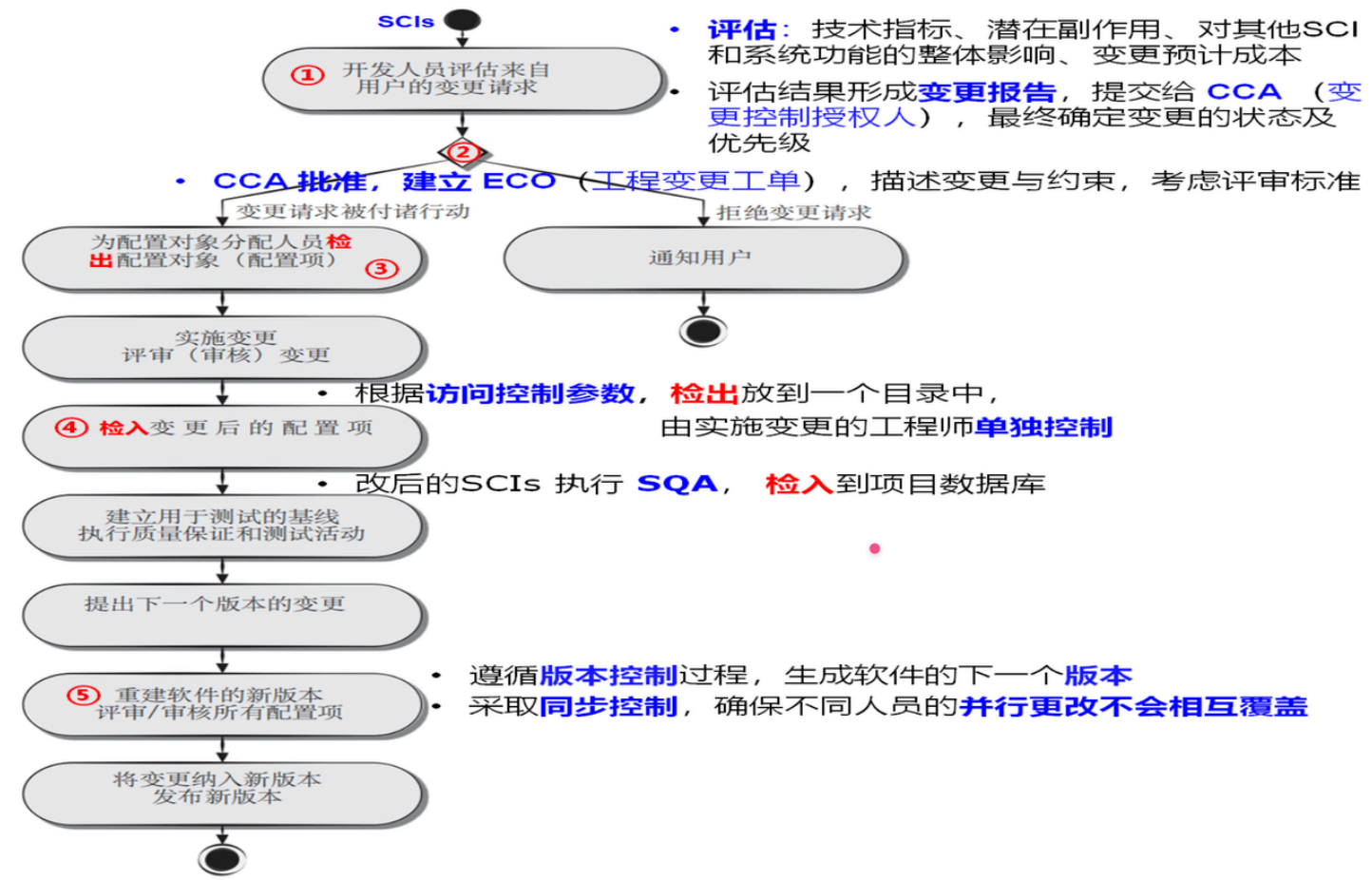
正式的变更通知 P334-335 划重点

Change Control Process

What are the **steps** in the change control process?

1. **evaluate** a change request, and **submit** a change report to **CCA** (change control authority). The **CCA** makes the final determination as to the **status** and **priority** of the change.
2. An **ECO** (engineering change order) is generated for each **approved change**.
3. Change SCIs are **checked out** from the project database subject by its access control **parameters**.
4. The **modified SCIs** is subjected to **SQA** procedures and **checked into** the project database.
5. **Version control** procedures are followed to produce the **next version** of the software. Adopting **Synchronization control** ensures that **parallel changes** made by different people do **not overwrite** with each other.

PPT上变更控制过程1



PPT上变更控制过程2

1. **评估** 变更请求，并向 **CCA（变更控制权限机构 / change control authority）提交**一份变更报告。CCA 对变更的**状态**和**优先级**做出最终决定。
2. 为每个**经批准的变更**生成一份 **ECO（工程变更单 / engineering change order）**。
3. 根据访问控制**参数**的限制，从项目数据库中**检出**需要变更的 SCI（软件配置项）。
4. 对**修改后的 SCI** 执行 **SQA（软件质量保证）程序，并将其检入**项目数据库。
5. 遵循**版本控制**程序以生成软件的**下一个版本**。采用**同步控制**可以确保不同人员进行的**并行变更不会相互覆盖**。

根据项目的推进，变更控制的严格程度分为三个层次：

1. **基线之前（非正式控制）**：在配置对象（SCI）成为基线之前，只需要进行非正式的变更控制，开发者可以较快且非正规地进行合理变更。
2. **基线之后（项目级控制）**：一旦配置对象通过正式技术评审并被批准成为基线，就必须实现项目级变更控制。此时：
   - 若变更是“局部的”，必须得到**项目管理者**的批准；
   - 若变更会影响到其他 SCI，则必须得到变更控制授权人（CCA）的批准。
3. **交付客户时（正式控制）**：当软件产品正式交付给客户时，正式的变更控制就全面开始实施。

5.2 基线

Baselines

The IEEE defines a **baseline** as:

- A **specification** or **product** that has been **formally reviewed** and **agreed** upon, that thereafter serves as the basis for further development, and that can be **changed** only through **formal change control** procedures.

A baseline is a **milestone** in the development of software

- that is marked by the delivery of one or more software configuration items and
- the approval of these SCI that is obtained through a formal technical review.

课件 基线的定义 划重点

5.3 软件配置项 P334

What is impact management ?

A web of software work product interdependencies must be considered when a change is made, it is necessary to properly understand these interdependencies and control their effects on other SCIs.

课堂作业二7.4 什么是影响管理P337 划重点

6. 第六部分 软件项目管理（第5，19，20，21章）

6.1 软件项目管理范围 4P P371

有效的软件项目管理集中于 4P，即人员 (People)、产品(Product)、过程 (Process)和项目 (Project)。

这四个要素的顺序并非随意排列。 P369

- 忽视**人员**因素的管理者将永远无法在管理中取得成功。
- 不鼓励**产品**开发早期沟通的管理者面临构建错误解决方案的风险。
- 不关注**过程**的管理者面临插入技术方法时产生混乱的风险。
- 没有坚实**项目**计划就开始工作的管理者会危及项目的最终成功

6.2 W5HH 原则 P377

1. **Why is the system being developed? 系统为什么要开发?**

2. **What will be done? 将要做什么?**
3. **When will it be done? 什么时候做?**
4. **Who is responsible for a function? 谁负责某个功能?**
5. **Where are they located organizationally? 他们在组织中的位置在哪里?**
6. **How will the job be done technically and managerially? 技术上和管理上如何完成工作?**
7. **How much of each resource is needed? 每种资源需要多少?**

> 1、In an excellent paper on software process and projects, Barry Boehm [Boe96] states: "[Y]ou need an organizing principle that scales down to provide simple [project] plans for simple projects." Boehm suggests an approach that addresses project objectives, milestones and schedules, responsibilities, management and technical approaches, and required resources. **How do we define key project characteristics?** （15 pts）

21-22-2双语真题

6.3 资源估计 P384

三类主要软件工程资源：人力资源 可复用软件资源 环境资源

6.4 风险评估 P404

1. 风险评估的核心定义 P404

风险预测旨在从两个方面对每个风险进行等级评定：

1. 风险发生的**可能性或概率**。
2. 如果风险发生，与该风险相关的**问题所产生的后果**。

2. 风险评估的四个步骤 P404

1. 建立一个尺度，反映风险被感知的可能性。
2. 描绘风险的后果。
3. 估算风险对项目及产品的影响。
4. 评估风险预测的整体准确性，以避免误解。

3. 评估工具：风险表 (Risk Table) P404-405

- **方法：** 团队首先列出所有识别出的风险，然后为每个风险估算其概率和影响等级。
- **排序：** 根据概率和影响的乘积进行排序，高概率且高影响的风险（位于 **Figure 21.2** 中的管理关注区）将成为风险管理的重点。
- **截断线 (Cutoff line)：** 项目经理会划定一条线，只有位于该线之上的风险才需要进行主动管理（制定 RMMM 计划）。

4. 评估风险影响 (Assessing Risk Impact) P406

评估风险后果时需要考虑三个因素：

- **性质 (Nature)：** 风险发生时可能产生的问题类型。
- **范围 (Scope)：** 风险的严重程度及其在整个项目中的分布情况。
- **时间 (Timing)：** 何时能感受到风险的影响以及持续多久。

5. 风险显露度 (Risk Exposure, RE) P406-407

为了量化风险，通常会计算**风险显露度**，计算公式如下： $RE = P \times C$

- **P**：风险发生的概率。
- **C**：风险发生时对项目造成的总成本（如：因开发成本增加而产生的额外费用）。

6.5 全球化团队 P80

见作业二的UNIT9

✍️ **软件工程师的素质（P75）**

Traits of Successful Software Engineers

要拓展

What are the characteristics of software engineers should possess?

- Sense of individual responsibility
- Has an acute awareness of the needs of other team members
- Brutal honesty
- Resilience under pressure
- Heightened sense of awareness
- Attention to detail
- Takes a pragmatic approach to software engineering

成功软件工程师的特质

软件工程师应该具备哪些特征?

- 个人责任感
- 对其他团队成员需求的敏锐意识
- 直言不讳
- 在压力下的韧性
- 高度的警觉性

- 注重细节
- 采取务实的软件工程方法

📍 **高效团队的特征（P77第二段到第四段）**

Effective Team Attributes

此题也要扩展

What are the key attributes of effective software teams?

- Sense of purpose.
- Sense of involvement.
- Sense of trust.
- Sense of improvement.
- Diversity of team member skill sets.

中文翻译

有效的软件团队的关键属性是什么?

- 目标感。
- 参与感。
- 信任感。
- 改进意识。
- 团队成员技能的多样性。

6.6 敏捷团队

1、1963～1966 年，开发的 IBM 360 机操作系统共 4000 多个模块，约 100 万条指令，投入 5000 人年，耗资数亿美元，结果延期交付后系统中仍发现大量（ 2000 个以上）的错误，项目负责人 F. D. Brooks 总结教训时提到：问题主要原因是管理出了问题，团队构建发生了问题。

请根据所学知识，简析选择团队机构应考虑什么因素 （7 分）？并谈一下个人对构建团队的认识（8 分）

21-22-2中文真题